Characterizing FNC1 by polynomial and bounded discrete ODE schemas

Melissa Antonelli, Arnaud Durand, Juha Kontinen

arxiv:2506.23404; CC BY 4.0

Editorial note. Full original author source and all detailed toappendix proofs retained. Appendix blocks render inline rather than being relocated by apxproof. Only the selected independent main characterization counts; immediate TC0 byproduct and existing FAC0 theorem do not.

Editorial note. Original duplicated lemma:logitadd labels in detailed proof/context renamed uniquely in render source. No references target these labels; original TeX retained.

Allowed standard prerequisites (editorial).

- Author-cited discrepancy lifting and standard resolution background. All source-local constructions and appendix proofs retained.

Abstract

In this paper, we provide a uniform framework for investigating small circuit classes and bounds through the lens of ordinary differential equations (ODEs). Following an approach recently introduced to capture the class of polynomial-time computable functions via ODE-based recursion schemas and later applied to the context of functions computed by unbounded fan-in circuits of constant depth ($\mathbf{FAC}^0$), we study multiple relevant small circuit classes. In particular, we show that *natural* restrictions on linearity and derivation along functions with specific growth rate correspond to kinds of functions that can be proved to be in various classes, ranging from $\mathbf{FAC}^0$ to $\mathbf{FAC}^1$. This reveals an intriguing link between constraints over linear-length ODEs and circuit computation, providing new tools to tackle the complex challenge of establishing bounds for classes in the circuit hierarchies and possibly enhancing our understanding of the role of counters in this setting. Additionally, we establish several completeness results, in particular obtaining the first ODE-based characterizations for the classes of functions computable in constant depth with unbounded fan-in and Mod 2 gates ($\mathbf{FACC}[\mathbf{2}]$) and in logarithmic depth with bounded fan-in Boolean gates ($\mathbf{FNC}^1$).

1 Introduction

Implicit computational complexity is a active area of theoretical computer science that aims to provide machine-independent characterizations of relevant complexity classes. In the context of recursion theory, a foundational work in this field is by Cobham [17], who, in the 1960s, provided an implicit characterization of the class of polynomial-time computable functions (**FP**) through a *limited* recursion schema. This work not only led to the development of function algebras characterizing multiple classes different from poly-time (see [14]), but also inspired several alternative recursion-theoretic approaches [5, 27, 29]. Among them, the one introduced in [9], shows that ordinary differential equations (ODEs) provide a natural tool for algorithmic design and offers an original characterization of **FP**, using *linear* systems of equations and *derivation along a logarithmically growing function* (such as the length function ℓ). Informally, the latter condition controls the number of recursion steps, while linearity governs the growth of objects generated during the computation. This approach has recently been generalized to the continuous setting [6, 7] and to the study of two small circuit classes [2]. In this paper, we extend the latter far further by presenting a *comprehensive* investigation of small circuit classes through the lens of ODE schemas.

Although small circuit classes have been characterized in different ways, investigating them through the ODE framework has never been attempted (except for [2]). This approach is both *interesting*, as for a descriptive method based on ODEs to be meaningful and fruitful, it must be able to adjust with very subtle changes in restricted models of computation, and *challenging*, because even simple and useful mathematical functions may not be computable within the considered classes and, consequently, the tools available may seem drastically restricted and the naturalness of the approach questionable. However, the results of this paper show that this is not hopeless. Very simple and natural restrictions on linearity – namely, constraining the values of coefficients and allowing (or not) the call for the sign of the defined function inside the linear term – and on the function along which derivation is performed, allows us to capture the very essence of small circuit computation. Many classical functions – such as polylog bit sum, parity, majority, iterated sum and product, and binary search – are shown to be naturally encapsulated in very simple linear ODE families. For this reason, we believe that our approach to this line of research, which is still in its beginning, can shed some new light on circuit computation and offer a simple (and machine-independent) framework for analyzing these notoriously hard problems through very *natural* and *uniform* constraints on linear ODEs.

Concretely, the results of this paper have led to completely original ODE-based characterizations for the classes of functions computable by unbounded fan-in circuits of constant depth and polynomial size that allow Mod 2 gates (**FACC**[**2**]) and by bounded fan-in circuits of logarithmic depth and polynomial size ($\mathbf{FNC}^1$). We have also improved our understanding of schemas capturing the classes of functions computable by unbounded fan-in circuits of constant depth and polynomial size ($\mathbf{FAC}^0$), possibly including Maj gates

($\mathbf{FTC}^0$), by generalizing and making them part of a more general picture. Finally, we have shown that considering schemas with *derivative along* $\ell_2 = \ell \circ \ell$ is definitely worth investigating: in particular, the corresponding linear schema is in $\mathbf{FNC}^1$, somewhat mirroring what ℓ-ODE does for **FP**.

Structure of the paper. In Section 2, we present the basics of the method developed in [9], the necessary notions to formulate our results, and recap definitions of the small circuit classes we will consider throughout the paper. In Section 3, we present, in a high-level yet systematic and uniform manner, the main schemas (differently) corresponding to the classes that will be studied. In Section 4, we introduce ODE-based schemas obtained by deriving along ℓ and imposing different constraints on linearity, and we show that they are computable in the respective circuit classes. We also provide improved completeness results for $\mathbf{FAC}^0$ and $\mathbf{FTC}^0$, along with completely original ODE-based characterizations for $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FNC}^1$. In Section 5, we introduce a new class of ODE schemas, the defining feature of which consists of deriving along ℓ_2. Our main result here is introduction of a very natural linear ℓ_2-ODE the computation of which can be performed in $\mathbf{FNC}^1$. We conclude in Section 6 pointing to possible directions for future research. For space reasons, most proofs are omitted or only sketched in the paper, but full details can be found in the Appendix. To make the exposition shorter, most of our characterizations are proved indirectly, as these proofs tend to be more compact. However, direct proofs (both in the uniform and non-uniform settings) can be provided (but are not presented here).

2 Preliminaries

We assume the reader familiar with the basics of complexity theory. In this section, we briefly introduce the approach to complexity delineated in [10] (see the original paper for full details) and a few notions needed for our new characterizations. We also cursorily recall the definitions of the circuit classes that we will consider in the remainder of the paper.

2.1 Capturing Complexity via ODEs

One of the main approaches developed in implicit computational complexity is that based on recursion. It was initiated by Cobham [17], who gave the first characterization for **FP** via a limited recursion schema. This groundbreaking result has inspired not only similar characterizations for other classes, but also alternative recursion-based approaches to capture poly-time computable functions, such as safe recursion [5] and ramification [27, 28]. Among them, the proposal by [9] offers the advantages of not imposing any explicit bound on the recursion schema or assigning specific role to variables. Instead, it is based on Discrete Ordinary Differential Equations (a.k.a. Difference Equations) and combines two novel features, namely derivation along specific functions, to control the number of

computation steps, and a special syntactic form of the equation (linearity), to control the object size.

Recall that the *discrete derivative of* $\mathbf{f}(x)$ is defined as $\Delta\mathbf{f}(x) = \mathbf{f}(x+1) - \mathbf{f}(x)$, and that ODEs are expressions of the form

$$\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial x} = \mathbf{h}\big(x, \mathbf{y}, \mathbf{f}(x,\mathbf{y})\big)$$

where $\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial x}$ stands for the derivative of $\mathbf{f}(x,\mathbf{y})$ considered as a function of x, for a fixed value for $\mathbf{y}$. When some initial value $\mathbf{f}(0,\mathbf{y}) = \mathbf{g}(\mathbf{y})$ is added, this is called Initial Value Problem (IVP). Then, in order to deal with complexity, some restrictions are needed. First, we introduce the notion of *derivation along functions*.

Notation 1. For $x \neq 0$, let $\ell(x)$ denote the length function, returning the length of x written in binary, i.e. $\lceil \log_2(x+1) \rceil$, and such that $\ell(0) = 0$. We use $\ell_2 = \ell \circ \ell$. For $u \geq 0$, let $\alpha(u) = 2^u - 1$ denote the greatest integer whose length is u; similarly, $\alpha_2(u) = 2^{2^u - 1} - 1$ is the greatest integer t such that $\ell_2(t) = u$. Finally, let $\div 2$ denote integer division by 2, i.e. for all $x \in \mathbb{Z}, x \div 2 = \lfloor \frac{x}{2} \rfloor$.

Notice that the value of $\ell(x)$ changes (it increases by 1) when x goes from $2^t - 1$ to 2^t (for $t \geq 1$). Similarly, the value of $\ell_2(x)$ changes when x goes from $2^{2^1-1} - 1$ to 2^{2^t-1} (for $t \geq 1$).

Definition 1 (ℓ-ODE Schema)**.** Let $\mathbf{f}, \lambda : \mathbb{N}^p \times \mathbb{Z} \to \mathbb{Z}$ and $\mathbf{h} : \mathbb{N}^{p+1} \to \mathbb{Z}$ be functions. Then,

$$\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial \lambda} = \frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial \lambda(x,\mathbf{y})} = \mathbf{h}\big(x, \mathbf{y}, \mathbf{f}(x,\mathbf{y})\big) \tag{1}$$

is a formal synonym of $\mathbf{f}(x+1,\mathbf{y}) = \mathbf{f}(x,\mathbf{y}) + \big(\lambda(x+1,\mathbf{y}) - \lambda(x,\mathbf{y})\big) \times \mathbf{h}\big(x, \mathbf{y}, \mathbf{f}(x,\mathbf{y})\big)$. When $\lambda(x,\mathbf{y}) = \ell(x)$, we call (1) a *length-ODE* or ℓ-ODE.

Intuitively, one of the key properties of the λ-ODE schema is its dependence on the number of distinct values of λ, i.e., the value of $\mathbf{f}(x,\mathbf{y})$ changes only when the value of $\lambda(x,\mathbf{y})$ does. Computation of solutions of λ-ODEs have been fully described in [10]. Here, we focus on the special case of $\lambda = \ell$, which is particularly relevant for our characterizations. If $\mathbf{f}$ is a solution of (1) with $\lambda = \ell$ and initial value $\mathbf{f}(0,\mathbf{y}) = \mathbf{g}(\mathbf{y})$, then $\mathbf{f}(1,\mathbf{y}) = \mathbf{f}(0,\mathbf{y}) + \mathbf{h}\big(\alpha(0), \mathbf{y}, \mathbf{f}(\alpha(0),\mathbf{y})\big)$. More generally, for all x and $\mathbf{y}$: $\mathbf{f}(x,\mathbf{y}) = \mathbf{f}(x-1,\mathbf{y}) + \Delta\big(\ell(x-1)\big) \times \mathbf{h}\big(x-1, \mathbf{y}, \mathbf{f}(x-1,\mathbf{y})\big) = \mathbf{f}\big(\alpha(\ell(x)-1), \mathbf{y}\big) + \mathbf{h}\big(\alpha(\ell(x)-1), \mathbf{y}, \mathbf{f}(\alpha(\ell(x)-1),\mathbf{y})\big)$, where $\Delta(\ell(t-1)) = \ell(t) - \ell(t-1)$. Starting from $t = x \geq 1$ and taking successive decreasing values of t, the first difference such that $\Delta(t) \neq 0$ is given by the biggest $t-1$ such that $\ell(t-1) = \ell(x) - 1$, i.e. $t - 1 = \alpha(\ell(x) - 1)$. Setting $\mathbf{h}(\alpha(-1), \mathbf{y}, \cdot) = \mathbf{f}(0,\mathbf{y})$, it follows by induction that:

$$\mathbf{f}(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} \mathbf{h}\big(\alpha(u), \mathbf{y}, \mathbf{f}(\alpha(u),\mathbf{y})\big)$$

In the present paper, both polynomial expressions (see [2]) and limited polynomial expressions are considered. Let $\mathsf{sg} : \mathbb{Z} \to \mathbb{Z}$ be the sign function over $\mathbb{Z}$, taking value 1 for $x > 0$ and 0 otherwise, and $\mathsf{cosg} : \mathbb{Z} \to \mathbb{Z}$ be the cosign function, such that $\mathsf{cosg}(x) = 1 - \mathsf{sg}(x)$.

Definition 2. A *limited* (sg-)*polynomial expression* is an expression over the signature $\{+, -, \div 2\}$ (and the sg function, resp.) on the set of variables/terms $X = \{x_1, \ldots, x_h\}$ and integer constants; (sg-)*polynomial expressions* are defined analogously over the signature $\{+, -, \div 2, \times\}$.

To define the notion of degree for polynomial expressions, we follow the generalization to sets of variables introduced in [2].

Definition 3. Given a list of variables $\mathbf{x} = x_{i_1}, \ldots, x_{i_m}$, for $i_1, \ldots, i_m \in \{1, \ldots, h\}$, the *degree of a set* $\mathbf{x}$ *in a* sg-*polynomial expression* P, $\deg(\mathbf{x}, P)$, is inductively defined as follows:

- $\deg(\mathbf{x}, x_{i_j}) = 1$, for $x_{i_j} \in \{x_{i_1}, \ldots, x_{i_m}\}$, and $\deg(\mathbf{x}, x_{i_j}) = 0$, for $x_{i_j} \notin \{x_{i_1}, \ldots, x_{i_m}\}$
- $\deg(\mathbf{x}, \mathsf{sg}(P)) = \deg(t\mathbf{x}, c) = 0$, for c being an integer constant
- $\deg(\mathbf{x}, P + Q) = \deg(\mathbf{x}, P - Q) = \max\{\deg(\mathbf{x}, P), \deg(\mathbf{x}, Q)\}$
- $\deg(\mathbf{x}, P \times Q) = \deg(\mathbf{x}, P) + \deg(\mathbf{x}, Q)$ and $\deg(\mathbf{x}, P \div 2) = \deg(\mathbf{x}, P)$.

A sg-polynomial expression P is said to be *essentially constant in a set of variables* $\mathbf{x}$ when $\deg(\mathbf{x}, P) = 0$ and *essentially linear in* $\mathbf{x}$, when $\deg(\mathbf{x}, P) = 1$.

Example 1. Let $\mathbf{x} = \{x_1, x_2, x_3\}$. The expression $P(x_1, x_2, x_3) = 3x_1x_3 + 2x_2x_3$ is essentially linear in x_1, in x_2 and in x_3. It is not essentially linear in $\mathbf{x}$, as $\deg(P, \mathbf{x}) = 2$. The expression $P'(x_1, x_2, x_3) = x_1\mathsf{sg}((x_1 - x_3) \times x_2) + x_2^3$ is essentially linear in x_1, essentially constant in x_3 and not linear in x_2. Clearly, P' is not linear in $\mathbf{x}$.

The concept of *linearity* allows us to control the growth of functions defined by ODEs. In what follows, we will consider functions $\mathbf{f} : \mathbb{N}^{p+1} \to \mathbb{Z}^d$, i.e. vectors of functions $\mathbf{f} = f_1, \ldots, f_d$ from $\mathbb{N}^{p+1}$ to $\mathbb{Z}$, and introduce the linear ℓ-ODE schema.

Definition 4 (Linear λ-ODE)**.** Given $\mathbf{g} : \mathbb{N}^p \to \mathbb{N}^d, \mathbf{h}, \lambda : \mathbb{N}^{p+1} \to \mathbb{Z}$ and $\mathbf{u} : \mathbb{Z}^{d+1} \times \mathbb{N}^{p+1} \to \mathbb{Z}^d$, the function $\mathbf{f} : \mathbb{N}^{p+1} \to \mathbb{Z}^d$ is *linear* λ*-ODE* definable from $\mathbf{g}, \mathbf{h}$ and $\mathbf{u}$ if it is the solution of the IVP with initial value $\mathbf{f}(0, \mathbf{y}) = \mathbf{g}(\mathbf{y})$ and such that:

$$\frac{\partial f(x, \mathbf{y})}{\partial \lambda} = \mathbf{u}\big(\mathbf{f}(x, \mathbf{y}), \mathbf{h}(x, \mathbf{y}), x, \mathbf{y}\big)$$

where $\mathbf{u}$ is *essentially linear in the list of terms* $\mathbf{f}(x, \mathbf{y})$. For $\lambda = \ell$, such schema is called *linear length ODE*.

If $\mathbf{u}$ is *essentially linear in* $\mathbf{f}(x, \mathbf{y})$, there exist matrices $\mathbf{A}$ and $\mathbf{B}$, whose coefficients are essentially constant in $\mathbf{f}(x, \mathbf{y})$, such that $\mathbf{f}(0, \mathbf{y}) = \mathbf{g}(\mathbf{y})$ and:

$$\frac{\partial \mathbf{f}(x, \mathbf{y})}{\partial \ell} = \mathbf{A}\big(x, \mathbf{y}, \mathbf{h}(x, \mathbf{y}), \mathbf{f}(x, \mathbf{y})\big) \times \mathbf{f}(x, \mathbf{y}) + \mathbf{B}\big(x, \mathbf{y}, \mathbf{h}(x, \mathbf{y}), \mathbf{f}(x, \mathbf{y})\big).$$

Then, for all x and $\mathbf{y}$, $\mathbf{f}(x, \mathbf{y})$ is

$$\sum_{u=-1}^{\ell(x)-1} \left(\prod_{t=u+1}^{\ell(x)-1} \Big(1 + \mathbf{A}\Big(\alpha(t), \mathbf{y}, \mathbf{h}(\alpha(t), \mathbf{y}), \mathbf{f}(\alpha(t), \mathbf{y})\Big)\Big) \right) \times \mathbf{B}\big(\alpha(u), \mathbf{y}, \mathbf{h}(\alpha(u), \mathbf{y}), \mathbf{f}(\alpha(u), \mathbf{y})\big)$$

with the convention that $\prod_x^{x-1} \kappa(x) = 1$ and $(\alpha(-1), \mathbf{y}, \cdot, \cdot) = \mathbf{f}(0, \mathbf{y})$. Actually, the use of matrices is not necessary for the scope of this paper, but it is consistent with the general presentation of computation through essentially linear ODEs, of which full details can be found in [9, 10].

One of the main results of [9] is the implicit characterization of **FP** by the algebra made of basic functions $\mathsf{0}, \mathsf{1}, \pi_i^p, \ell, +, -, \times, \mathsf{sg}$, where π_i^p denotes the projection function, and closed under composition ($\circ$) and ℓ-ODE.

2.2 On Small Circuit Classes and Function Algebras

Boolean circuits are vertex-labeled directed acyclic graphs whose nodes/gates are either input nodes, output nodes or are labeled with a Boolean function from the set $\{\wedge, \vee, \neg\}$. Boolean circuit with modulo gates allows in addition gates labeled Mod 2 or, more generally, Mod p, that output the sum of the inputs modulo 2 or p, respectively. Finally, Boolean circuit with majority gates includes Maj gates, that output 1 when the majority of the inputs are 1's. A family of circuits is **Dlogtime**-uniform if there is a Turing machine (with a random access tape) that decides in deterministic logarithmic time the direct connection language of the circuit, i.e. which, given $1^n, a, b$ and $t \in \{\wedge, \vee, \neg\}$, decides if a is of type t and b is a predecessor of a in the circuit (and analogously for input/output nodes). When dealing with circuits, the resources of interests are *size*, i.e. the number of gates, and *depth*, i.e. the length of the longest path from input to output (see [33] for further details and related results).

Definition 5 (Classes $\mathbf{FAC}^i$ and $\mathbf{FNC}^i$)**.** For $i \in \mathbb{N}$, $\mathbf{AC}^i$ (resp., $\mathbf{NC}^i$) is the class of language recognized by a **Dlogtime**-uniform family of Boolean circuits of unbounded (resp. bounded) fan-in gates of polynomial size and depth $O((\log n)^i)$. We denote by $\mathbf{FAC}^i$ (resp., $\mathbf{FNC}^i$) the corresponding function class.

It is known that unbounded (poly(n)) fan-in can be simulated using a bounded tree of depth $O(\log n)$, so that $\mathbf{FNC}^i \subseteq \mathbf{FAC}^i \subseteq \mathbf{FNC}^{i+1}$. However, the inclusion has been proved to be strict only for $i = 0$. In contrast, when dealing with classes with counting ability, interesting bounds have been established.

Definition 6 (Classes **FACC**[**p**] and **FTC**^{0})**.** The class **ACC**[**p**] (resp., **TC**^{0}) is the class of languages recognized by a **Dlogtime**-uniform family of Boolean circuits including MOD p (resp., MAJ) gates of polynomial size and constant depth. We denote by **FACC**[**p**] (resp., **FTC**^{0}) the corresponding function class.

We can even consider other levels in the **FTC** hierarchy, such that for $i \in \mathbb{N}$, **FTC**^{i} corresponds to classes of functions computed by families of unbounded fan-in circuits including MAJ gates, of polynomial size and depth $O((\log\ n)^i)$. It is known that the inclusion **FAC**$^0 \subset$ **FACC**[**2**] is proper, as the parity function cannot be computed in constant depth by standard unbounded fan-in Boolean gates, but it can be if MOD 2 gates are added. Moreover, since any MOD p gate can be expressed by MAJ, **FACC**[**p**] $\subset$ **FTC**^{0}.

As mentioned, Cobham's work paved the way to several recursion-theoretic characterizations for classes other than **FP**, including small circuit ones. In 1988/90, Clote introduced the algebra $\mathcal{A}_0$ to capture functions in the log-time hierarchy (equivalent to **FAC**^{0}) using so-called *concatenation recursion on notation* (CRN) [11, 12]. The greater classes **FAC**^{i} and **FNC** are captured adding *weak bounded recursion on notation* [12], while **FNC**^{1} has been characterized both in terms of the *k-bounded recursion on notation* (k-BRN) schema [13] and relying on the basic function *tree* [12]. In 1990, an alternative algebra (and logic), based on a schema called *upward tree recursion*, was introduced to characterize **FNC**^{1} [18]. In 1991, a new recursion-theoretic characterization (and arithmetic *à la Buss*) for **FNC** was presented [1]. In 1992 and 1995, other small circuit classes, including **FTC**^{0}, were considered (and re-considered) by Clote and Takeuti, who also introduced bounded theories for them [15, 16]. More recently, characterizations for **FNC**^{i} were developed in [8], this time following the ramification approach. In [2], the first ODE-based characterizations for **FAC**^{0} and **FTC**^{0} were introduced. Related approaches to capture small circuit classes have also been provided in the framework of model- and proof-theory, both in the first and second order [25, 4, 23, 30, 18, 1, 15, 16, 26, 20, 3, 19, 21]. To the best of our knowledge, so far, no *uniform* implicit framework to characterize these classes has been offered in the literature.

3 Towards a Uniform Understanding of Small Circuit Classes

We informally outline here the comprehensive investigation of small circuit classes in terms of ODEs (see Figure 1 for a complete description). By slightly modifying the restrictions on the linearity allowed in ℓ-ODE and ℓ_2-ODE, we can perform computation and, in several cases, characterize multiple function classes in the range from **FAC**^{0} to **FAC**^{1}.

Notation 2. An equation or, by extension, an ODE schema is said to be *strict* when it does not include any call to $f(x, \mathbf{y})$ in A or B (not even under the scope of the sign function). We say that B includes *simple* calls to $f(x, \mathbf{y})$ if $f(x, \mathbf{y})$ occurs in B only in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$.
For clarity's sake, we will use $k(x, \mathbf{y})$ for functions with limited range, typically in $\{0, 1\}$,

and $K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y}))$ for (possibly limited) sg-polynomial expressions of restricted range.
We use $s\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$ or (when not ambiguous) simply $s(x)$ as an abbreviation for signed expressions in which $f(x, \mathbf{y})$ can occur, i.e. $s = \mathsf{sg}\big(E(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y}))\big)$, where E is any polynomial expression in which $x, \mathbf{y}, h(x, \mathbf{y})$ and $f(x, \mathbf{y})$ may appear. Finally, for readability, we often use

- $s(t)$ as a shorthand for $s\big(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y})\big)$,
- $K(t, \mathbf{y}, h, f)$, or even $K(t)$, as a shorthand for $K(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y}))$,
- $K_i(t)$ (or $B_i(t)$), with $i \in \{0, 1\}$, as a shorthand for $K(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y}))$ in which $s(\alpha(t)) = i$.

When different signed expressions $s_1, \ldots, s_r$ occur in A or B these notational conventions are generalized accordingly. If not stated otherwise, we assume that expressions used in $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$ schemas are limited sg-polynomial expressions (since multiplication is not in these classes), while those in $\mathbf{FNC}^1$ and $\mathbf{FAC}^1$ are sg-polynomial expressions.

Let us first examine the evolution in the expressive power of the linear schema (see Def. 4), which captures **FP**,

$$A(x, \mathbf{y}, h, f) \times f(x, \mathbf{y}) + B(x, \mathbf{y}, h, f)$$

when calls to f are not allowed, i.e. when dealing with equation of the form $A(x, \mathbf{y}, h) \times f(x, \mathbf{y}) + B(x, \mathbf{y}, h)$, but still deriving along ℓ. In this case, f can be expressed as a combination of iterated multiplications and products and is in $\mathbf{FTC}^0$. Interestingly, this schema is also the key ingredient to capture $\mathbf{FTC}^0$ and to provide a corresponding completeness result. On the other hand, if calls to f are allowed (i.e. we are back to the general form), but the derivative is along the "slower" function ℓ_2, then expressivity jumps to $\mathbf{FNC}^1$.

As a second example to focus on the smallest classes, consider schemas of the form $-f(x, \mathbf{y}) + B(x, \mathbf{y}, h, f)$, for $B \geq 0$, with derivative along ℓ and only *simple* calls to f. We show that such defined functions are in $\mathbf{FNC}^1$. Forcing the second part to be in $\{0, 1\}$, i.e. considering equations of the forms $-f(x, \mathbf{y}) + K(x, \mathbf{y}, h, f)$ and $-f(x, \mathbf{y}) + k(x, \mathbf{y}, h)$ (namely, in the latter case, calls to f are not allowed) makes the functions respectively in $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FAC}^0$. Remarkably, in the first two cases, the mentioned schemas are central to characterize the corresponding classes.

Finally, we examine (strict) equations of the form $B(x, \mathbf{y}, h)$ (where B is a limited sg-polynomial expression), this time deriving along ℓ_2. Equations of this form provide the ODE equivalent to log iterated sums and, not surprisingly, define a computation in $\mathbf{FAC}^0$. Generalizing them to the non-strict setting by allowing equations of the form $B(x, \mathbf{y}, h, f)$, where calls to f occur, gives a really more tricky schema, that is shown computable in $\mathbf{FTC}^0$.

Figure 1: ODE Schemas and Small Circuit Classes in a Nutshell

	strict schemas along ℓ	non-strict schemas along ℓ	schemas along ℓ_2
$\mathbf{FAC}^0$	$(k(x)-1)\times f(x)$ $\pm f(x)+k(x)$	$(K(x,f)-1)\times f(x)$	$B(x)$
$\mathbf{FACC}[\mathbf{2}]$	$-k(x)\times f(x)+k(x)\times k'(x)$	$-f(x)+K(x,f)$	
$\mathbf{FTC}^0$	$A(x)\times f(x)+B(x)$	$K(x,f)$ $K(x,f)\times f(x)$	$B(x,f)$
$\mathbf{FNC}^1$		$f(x)+K(x,f)$ $-f(x)+B(x,f)$	$A(x,f)\times f(x)+B(x,f)$
$\mathbf{FAC}^1$		$B(x,f)$	

The table summarizes the features defining the equations of the main linear schemas in the corresponding small circuit class. For readability, the arguments h and $\mathbf{y}$ are omitted. Schemas in blue are not only computable in the corresponding class, but also provide a characterization for it. Calls in non-strict schemas deriving along ℓ are simple or restricted to $s=\mathsf{sg}(f(x)-c)$, for $c\in\mathbb{N}$, except for the schema in $\mathbf{FACC}[\mathbf{2}]$, and B is assumed to take positive values only.

4 Investigating Circuit Complexity Deriving along ℓ

In this section, we present linear ODE schemas obtained by deriving along ℓ and show that they are computable and, in some cases, capable of characterizing the corresponding class.

4.1 The Class $\mathbf{FAC}^0$

We start by revising the characterization of $\mathbf{FAC}^0$ from [2]. Recall that the function algebra $\mathbb{ACDL}$ capturing $\mathbf{FAC}^0$ is based on the two very restrictive forms of ℓ-ODE's below.

Definition 7 (Schemas ℓ-ODE$_1$, ℓ-ODE$_3$)**.** Given $g:\mathbb{N}^p\to\mathbb{N}$ and $k:\mathbb{N}^{p+1}\to\{0,1\}$, the function $f:\mathbb{N}^{p+1}\to\mathbb{N}$ is obtained by ℓ-ODE$_1$ from g, if it is the solution of the IVP defined by $f(0,\mathbf{y})=g(\mathbf{y})$ and

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell}=f(x,\mathbf{y})+k(x,\mathbf{y}).$$

On the other hand, f is obtained by ℓ-ODE$_3$ from g if it is the solution of the IVP defined by $f(0,\mathbf{y})=g(\mathbf{y})$ and

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell}=-\left\lceil\frac{f(x,\mathbf{y})}{2}\right\rceil.$$

Relying on them, in [2], the ODE-based function algebra below is introduced:

$$\mathbb{ACDL}=[0,1,\pi_i^p,\ell,+,-,\#,\div 2,\mathsf{sg};\circ,\ell\text{-ODE}_1,\ell\text{-ODE}_3]$$

where, as standard, $x\#y = 2^{\ell(x)\times\ell(y)}$. In the same paper, this class is shown to precisely characterize $\mathbf{FAC}^0$, see [2, Cor. 19]. Here, we dramatically simplify the (indirect) completeness proof, by showing that ℓ-ODE$_3$ precisely captures the idea of looking for the value of a bit in an $\ell(x)$ long sequence, while ℓ-ODE$_1$ corresponds to the idea of left-shifting (the binary representation of) a given number, possibly adding 1.

Remark 1 (The Function BIT)**.** Given an input y of length $\ell(y)$, for any $i \in \{0,\ldots,\ell(y)\}$, the schema ℓ-ODE$_3$ allows us to compute its ith bit in a direct way. Let us consider an instance of ℓ-ODE$_3$ such that $g(\mathbf{y}) = y$, i.e. defined by $\mathsf{rsh}(0,y) = y$ and

$$\frac{\partial \mathsf{rsh}(x,y)}{\partial \ell} = -\left\lceil \frac{\mathsf{rsh}(x,y)}{2} \right\rceil.$$

Then, the $\ell(x)^{th}$ bit of y is $\mathsf{BIT}(x,y) = \mathsf{rsh}(x,y) - \mathsf{rsh}(x+1,y) \times 2$.

Remark 2 (The CRN Schema)**.** In [12], CRN is defined by $f(0,\mathbf{y}) = g(\mathbf{y})$ and $f(s_i(x),\mathbf{y}) = s_{h_i(x,\mathbf{y})}(f(x,\mathbf{y}))$, for $i \in \{0,1\}$, h_i taking value in $\{0,1\}$ and $s_i(x) = 2x + i$ being binary successor functions. We can rewrite this schema as an instance of ℓ-ODE$_1$ such that

$$\begin{aligned}
f_{crn}(0,y,\mathbf{y}) &= g(\mathbf{y}) \\
\frac{\partial f_{crn}(x,y,\mathbf{y})}{\partial \ell} &= f_{crn}(x,y,\mathbf{y}) + h_{\mathsf{BIT}(\ell(y)-\ell(x)-1,y)}(x,\mathbf{y}) \\
&= f_{crn}(x,y,\mathbf{y}) + h_0(x,\mathbf{y}) \times \mathsf{cosg}\big(\mathsf{BIT}(\ell(y)-\ell(x)-1,y)\big) \\
&\qquad + h_1(x,\mathbf{y}) \times \mathsf{sg}\big(\mathsf{BIT}(\ell(y)-\ell(x)-1,y)\big)
\end{aligned}$$

where, by definition, for any t, $h_i(t,\mathbf{y}) \leq 1$. Then, $\mathsf{crn}(x,\mathbf{y}) = f_{crn}(x,x,\mathbf{y}) = f(x,\mathbf{y})$.

Theorem 1 ([2])**.** $\mathbf{FAC}^0 = \mathbb{ACDL}$.

Proof. ($\supseteq$) By [2, Prop. 12, 17]. ($\subseteq$) The proof is indirect and passes through Clote's algebra

$$\mathcal{A}_0 = [0, \pi_i^p, \mathsf{s}_0, \mathsf{s}_1, \ell, \mathsf{BIT}, \#; \circ, \mathrm{CRN}],$$

which, in [12], is proved to characterize $\mathbf{FAC}^0$. Indeed, its basic functions 0, π_i^p, ℓ, $\#$ and composition are by design in $\mathbb{ACDL}$, s_0, s_1 can be rewritten in our setting almost for free, while BIT and CRN can be encoded as in Remarks 1 and 2. □

4.2 Strict Schemas in $\mathbf{FAC}^0$

Lemma 1. *Let $k : \mathbb{N}^p \to \{0,1\}$ be computable in $\mathbf{FAC}^0$. Then, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined by k as the solution of the IVP below:*

$$\begin{aligned}
f(0,\mathbf{y}) &= g(\mathbf{y}) \\
\frac{\partial f(x,\mathbf{y})}{\partial \ell} &= -f(x,\mathbf{y}) + k(x,\mathbf{y})
\end{aligned}$$

is in $\mathbf{FAC}^0$.

Proof. By definition of ℓ-ODE, we simply have to compute $f(x, \mathbf{y}) = k(x, \mathbf{y})$, which can be done in $\mathbf{FAC}^0$, whenever $k(x, \mathbf{y})$ can be computed in $\mathbf{FAC}^0$. □

Lemma 2. *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $k : \mathbb{N}^{p+1} \to \{0, 1\}$ be computable in $\mathbf{FAC}^0$. Then, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined by k as the solution of the IVP below:*

$$f(0, \mathbf{y}) = g(\mathbf{y})$$
$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = \big(k(x, \mathbf{y}) - 1\big) \times f(x, \mathbf{y})$$

is in $\mathbf{FAC}^0$.

Proof. By definition of ℓ-ODE:

$$f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} k(\alpha(u), \mathbf{y})$$

with the convention that $k(\alpha(-1), \mathbf{y}) = g(\mathbf{y})$. Clearly $f(x, \mathbf{y}) = g(\mathbf{y})$ when $k(\alpha(t), \mathbf{y}) = 1$ for all $t \in \{0, \ldots, \ell(x)\}$ and 0 otherwise. By hypothesis, we can compute each $k(\alpha(t), \mathbf{y})$ in parallel and "iterated multiplication" by 0 (i.e., cancellation) and 1 (i.e., no change) can be performed in $\mathbf{FAC}^0$, i.e. by a single layer with unbounded fan-in $\wedge$. □

In the context of a broader understanding of the hierarchy between classes, we have also considered non-strict ℓ-ODE schemas that are computable in $\mathbf{FAC}^0$. As anticipated, this investigation demonstrates a strong link between depth constraints over circuits and the syntactical constraints that define the corresponding linear length ODE. Particularly interesting in this setting is the IVP defined by linear equations in which $A \in \{-1, 1\}$ and $B = 0$. As long as $f(x, \mathbf{y})$ occurs in $A(= K - 1)$ only in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, computation through this schema is proved to be in $\mathbf{FAC}^0$. Remarkably, computing the solution of such system corresponds to performing bounded search.

Lemma 3. *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^0$. Then, $f(x, \mathbf{y})$ defined from g and h as the solution of the IVP made of $f(0, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = \big(K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) - 1\big) \times f(x, \mathbf{y})$$

where $K \in \{0, 1\}$ is a limited sg-polynomial expression and $f(x, \mathbf{y})$ occurs in it only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, is in $\mathbf{FAC}^0$.

Proof Sketch. By definition of ℓ-ODE, $f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})\big)$, with the convention that $K(\alpha(-1), \cdot) = g(\mathbf{y})$. Clearly, if there is a $t \in \{0, \ldots, x\}$ such that $K(t) = 0$, then $f(x, \mathbf{y}) = 0$. Therefore, we compute $f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} K_1(u)$ which is either 0, if $g(\mathbf{y}) = 0$ or there is a $K_1(u) = 0$, or $g(\mathbf{y})$, otherwise. By hypothesis, we can compute in $\mathbf{FAC}^0$ both $g(\mathbf{y})$ and, for any $u \in \{0, \ldots, \ell(x) - 1\}$, $K_1(u)$. Thus, we conclude in one layer defined by an unbounded fan-in $\wedge$-gate. □

This result can actually be strengthened by showing that it holds even for generalized expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}) - r)$ occurring in K.

To conclude the presentation of schemas for $\mathbf{FAC}^0$, let us consider simple cases of bounded search encoded by the aforementioned ODE schemas. (For further details, see Appendix 4.13.)

Example 2 (BSearch)**.** Let $R \subseteq \mathbb{N}^{p+1}$ and h_R be its characteristic function. Then, for all x and $\mathbf{y}$, $(\forall z \leq \ell(x))R(z, \mathbf{y}) = \mathsf{sg}(f(x, \mathbf{y}))$, where f is defined by $f(0, \mathbf{y}) = \mathsf{sg}(h_R(0, \mathbf{y}))$ and $\frac{\partial f(x,\mathbf{y})}{\partial \ell} = (h_R(x, \mathbf{y}) - 1) \times f(x, \mathbf{y})$, which is clearly an (very limited) instance of the schema presented in Lemma 3. Another example of limited search consists in checking whether, at some point, $f(x, \mathbf{y})$ reaches or not a boundary, say r. This can be expressed in a natural way by an instance of the (generalized) non-strict schema (in $\mathbf{FAC}^0$) defined by the equation $\frac{\partial f(x,\mathbf{y})}{\partial \ell} = \big(\mathsf{sg}(f(x, \mathbf{y}) - r) - 1\big) \times f(x, \mathbf{y})$.

4.3 Non-Strict Schemas in $\mathbf{FAC}^0$

Lemma 4 (Generalized Lemma 3)**.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^0$. Then, $f(x, \mathbf{y})$ defined from g and h as the solution of the IVP made of $f(0, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = \big(K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) - 1\big) \times f(x, \mathbf{y})$$

where $K \in \{0, 1\}$ is a limited sg-polynomial expression and $f(x, \mathbf{y})$ occurs in it only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}) - c)$ for any constant $c \in \mathbb{N}$, is in $\mathbf{FAC}^0$.

Proof. Let $s_j(u) = \mathsf{sg}\big(f(\alpha(u), \mathbf{y}) - c_j\big)$, $j \in \{1, \ldots, h\}$ be the list of signed terms appearing in K and $c = \max_j c_j$. Let's assume that the list can be ordered so that, for any $j', j'' \in \{0, \ldots, h\}$ and $j'' > j'$, if $s_{j''} = 1$, then $s_{j'} = 1$ and vice versa, if $s_{j'} = 0$, then $s_{j''} = 0$. For readability, we use $K_{j'}(u)$ as a shorthand for $K_{i_0 \ldots i_{j'} i_{j'+1} \ldots i_j}(u) = K_{\underbrace{1 \ldots 1}_{j' \text{ times}} \underbrace{0 \ldots 0}_{j - j' \text{ times}}}(u)$ where, for any $j'' \in \{1, \ldots, j\}$, $i_{j''} \in \{0, 1\}$, corresponding to $K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y}))$ obtained by substituting $s_0, \ldots, s_{j'}$ with 0 and $s_{j'+1}, \ldots, s_j$ with 1. By definition of ℓ-ODE,

$$f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})\big)$$

with the convention that $K(\alpha(-1), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})) = g(\mathbf{y})$. Notice that if there is a t such that $K(t) = 0$, then $f(x, \mathbf{y}) = 0$. Then, $f(x, \mathbf{y})$ is either 0, when $g(\mathbf{y}) = 0$ or there is a t such that $K(t) = 0$, or $g(\mathbf{y})$, if, for all t, $K(t) = 1$. Since, for any $t \in \{0, \ldots, \ell(x)\}$, $f(\alpha(t), \mathbf{y})$ can only take two different values, namely 0 or $g(\mathbf{y})$, all the values of $K_j(t)$ (i.e., of conditional $f(\alpha(t+1), \mathbf{y})$ and the corresponding $s_j(t)$ can be computed independently from each other, in the beginning. Intuitively this is implemented in constant depth as follows:

- In parallel, compute $g(\mathbf{y})$ and, for any $u \in \{0, \ldots, \ell(x) - 1\}$, $K_0(t), \ldots, K_j(t)$ and $s_0(g(\mathbf{y})), \ldots, s_j(g(\mathbf{y}))$.
- In parallel, for each $j' \in \{0, \ldots, j\}$ and $t \in \{0, \ldots, \ell(x) - 1\}$, check the consistency between the initial value (i.e. what is the correct value of each $s_{j'}(g(\mathbf{y}))$) and that between all subsequent values of $K_{j'}(t), K_{j'}(t), \ldots$. This can be done via unbounded fan-in $\wedge$-gate.
- In one layer, check whether the correct (i.e. consistent with the initial value of $g(\mathbf{y})$) sequence of values corresponds to strictly positive $K(t)$ only. This is just the (unbounded) disjunction between all the previous results.

□

In the restrictive case of $s = \mathsf{sg}(f(x, \mathbf{y}))$ it is immediate to see that this (non-strict) schema can be rewritten in terms of (strict) ℓ-ODE$_1$ (offering an alternative indirect proof of Lemma 3). Indeed, since $s = \mathsf{sg}(f(x, \mathbf{y}))$, $f(x, \mathbf{y}) \neq 0$ when $g(\mathbf{y}) > 0$ and for any $u \in \{0, \ldots, \ell(x) - 1\}$, $K(u) = K_1(u) > 0$. This can be expressed via ℓ-ODE$_1$ as follows:

$$f'(0, \mathbf{y}) = \mathsf{cosg}(g(\mathbf{y}))$$
$$\frac{\partial f'(x, \mathbf{y})}{\partial \ell} = f'(x, \mathbf{y}) \times \mathsf{cosg}\big(K_1(x)\big).$$

Then, $\mathsf{bsearch}(x, x) = \mathsf{cosg}(f'(x, x))$. Thus, also this schema allows us to perform computation corresponding to bounded search, and in fact it can be rewritten in terms of weaker schema introduced in Lemma 2.

Remark 3. In the limited case of $s = \mathsf{sg}(f(x, \mathbf{y}))$, the given schema is actually equivalent to the one presented in Lemma 2:

- It is clear that the schema of Lemma 2 is nothing but a special case of the one of Lemma 3, such that $f(x, \mathbf{y})$ does not occur in K.
- The converse direction also holds. By definition,

  $$K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big) = K_0(x) \times \mathsf{cosg}\big(f(x, \mathbf{y})\big) + K_1(x) \times \mathsf{sg}\big(f(x, \mathbf{y})\big)$$

  Then,

  $$\begin{aligned}
  \frac{\partial f(x, \mathbf{y})}{\partial \ell} &= \big(K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) - 1\big) \times f(x, \mathbf{y}) \\
  &= \big(\big(K_0(x) \times \mathsf{cosg}(f(x, \mathbf{y})) + K_1(x) \times \mathsf{sg}(f(x, \mathbf{y}))\big) - 1\big) \times f(x, \mathbf{y}) \\
  &= K_0(x) \times \mathsf{cosg}(f(x, \mathbf{y})) \times f(x, \mathbf{y}) + K_1(x) \times \mathsf{sg}(f(x, \mathbf{y})) \times f(x, \mathbf{y}) - f(x, \mathbf{y}) \\
  &= K_1(x) \times f(x, \mathbf{y}) - f(x, \mathbf{y}) \\
  &= (K_1(x) - 1) \times f(x, \mathbf{y})
  \end{aligned}$$

  where $K_1(x)$ does not include any occurrence of s, i.e. $K_1(x) = k(x, \mathbf{y}) \in \{0, 1\}$, making the corresponding IVP an instance of the schema of Lemma 2.

4.4 The Class FACC[2]

By weakening the constraints on linearity of the schemas which define $\mathbf{FAC}^0$, we introduce an ℓ-ODE the computation of which is provably in $\mathbf{FACC}[\mathbf{2}]$ and actually characterizes this class. We consider a non-strict schema such that $A = -1$ and $B = K \in \{0, 1\}$. As we will see (Section 4.10), it is a special case of the schema we will introduce to capture $\mathbf{FNC}^1$. Together with the functions and schemas defining $\mathbb{ACDL}$, it is also enough to capture $\mathbf{FACC}[\mathbf{2}]$, thus providing the very first ODE-based characterization for this small circuit class "with counters".

Definition 8 (Schema ℓ-b_0ODE)**.** Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-b_0ODE if it is the solution of the IVP with $f(0, \mathbf{y}) = g(\mathbf{y})$ and

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = -f(x, \mathbf{y}) + K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where $K \in \{0, 1\}$ is a limited sg-polynomial expression and $f(x, \mathbf{y})$ occurs in K only under the scope of the sign function.

Notably, this schema allows us to capture the computation performed by Mod 2 gates and to rewrite the parity function in a very natural way.

Example 3 (The Parity Function)**.** Let z be a string of $\ell(z) = n$ bits. To count the number of 1's in z we consider the following system defined by $\mathsf{par}(0, y) = \mathsf{BIT}(0, y)$ and:

$$\frac{\partial \mathsf{par}(x, y)}{\partial \ell} = -\mathsf{par}(x, y) + \big(\mathsf{sg}(\mathsf{par}(x, y)) \times \mathsf{cosg}(\mathsf{BIT}(\ell(x) + 1, y)) + \mathsf{cosg}(\mathsf{par}(x, y)) \times \mathsf{sg}(\mathsf{BIT}(\ell(x) + 1, y))\big)$$

which is clearly an instance of ℓ-b_0ODE. The parity function is now computed by $\mathsf{par}(z, z)$.

Notice that this example shows that ℓ-b_0ODE really increases the power of ℓ-ODE_1, as the parity function is known not to be in $\mathbf{FAC}^0$ (see [22]). This also indicates that ℓ-b_0ODE is actually a schema capturing the specific computational power corresponding to $\mathbf{FACC}[\mathbf{2}]$. Moreover, it is shown that the desired closure property holds for it.

Lemma 5. *If $f(x, \mathbf{y})$ is defined by ℓ-b_0ODE from functions in $\mathbf{FACC}[\mathbf{2}]$, then $f(x, \mathbf{y})$ is in $\mathbf{FACC}[\mathbf{2}]$ as well.*

Proof Sketch. For simplicity, let us consider the simple case of $f(x, \mathbf{y})$ occurring only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$. Relying on the fact that $K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) = K_1(x) \times \mathsf{sg}(f(x, \mathbf{y})) + K_0(x) \times \mathsf{cosg}(f(x, \mathbf{y}))$, we can

rewrite the equation defining ℓ-b$_0$ODE as follows:

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell} = -f(x,\mathbf{y}) \times \big(K_0(x) = K_1(x)\big) + \big(K_0(x) = K_1(x)\big) \times \Big(\big((K_0(x) = 1) \times \textsc{Mod}\ 2\Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big)\Big) = 0\big) + \big((K_0(x) = 0) \times \textsc{Mod}\ 2\Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big)\Big) = 1\big)\Big).$$

Intuitively, if $K_0(t) \neq K_1(t)$, no change occurs. In contrast, if $K_0(t) = K_1(t)$ we restart the computation (as the preceding value of the input must not be the last one for which $K_0 = K_1$) by removing the previous value of $f(t,\mathbf{y})$, and return 1 if either one of the following holds:

i $K_0(t) = K_1(t) = 1$, i.e. $K(t)$ "generates" a 1, and the number of changes of values (temporarily assuming that t is the last value in $\{0,\ldots,\ell(x)-1\}$ such that $K_0(t) = K_1(t)$) between subsequent values, i.e. $K_0(t)$ and $K_0(t+1)$, is such that 1 (possibly alternating) "propagates" until the end of the computation or

ii $K_0(t) = K_1(t) = 0$, i.e. $K(t)$ "generates" a 0, and the subsequent number of changes of values is such that 0 does not propagate to the final value, i.e. $\textsc{Mod}\ 2\Big(\sum_{i=t}^{\ell(x)-1} \big(K_0(t) = K_0(t+1)\big)\Big) = 1$.

It is easy to see that the value computed in this way corresponds to the final value of the computation, i.e. $f(x,\mathbf{y})$ is given by $f(t,\mathbf{y})$ such that for any $t' \in \{t,\ldots,\ell(x)\}, K_0(t') \neq K_1(t')$. Clearly, this is a special case of ℓ-b$_0$ODE in which no call to $f(x,\mathbf{y})$ occurs in A or B, namely of the form $\frac{\partial f(x,\mathbf{y})}{\partial \ell} = -k(x,\mathbf{y}) \times f(x,\mathbf{y}) + k(x,\mathbf{y}) \times k'(x,\mathbf{y})$, for $k(x,\mathbf{y}), k'(x,\mathbf{y}) \in \{0,1\}$. Computation through this schema can be implemented by circuits of constant depth and polynomial size including $\textsc{Mod}\ 2$ gates: for any $t \in \{0,\ldots,\ell(x)-1\}$, we can compute $k(\alpha(t),\mathbf{y})$ and $k'(\alpha(t),\mathbf{y})$ in parallel and check both their mutual consistency, i.e. whether $k(\alpha(t),\mathbf{y}) = k'(\alpha(t),\mathbf{y})$, and the consistency between subsequent pairs, i.e. whether $k(\alpha(t),\mathbf{y}) = k(\alpha(t+1),\mathbf{y})$; then, using $\textsc{Mod}\ 2$ gates, we can compute whether the number of changes of values is even or odd and conclude in constant depth via (unbounded fan-in) $\vee$-gate. (For further details, see Appendix 4.6). □

4.5 Strict Schemas in FACC[2]

We consider a strict schema, which can be computed in **FACC**[**2**]. It weakens the constraints on A so that the coefficient of $f(x,\mathbf{y})$ can now be 0, but its value is related to that of B: by a special limited form of linearity, in which $A = -k(x,\mathbf{y}) \in \{0,1\}$ and $B = k(x,\mathbf{y}) \times k'(x,\mathbf{y}) \in \{0,1\}$, it is not possible that the coefficient of $f(x,\mathbf{y})$ is 0, when $B \neq 0$ (BCOUNT is not in **FACC**[**2**]).

Lemma 6. *Let $g : \mathbb{N}^p \to \{0,1\}$ and $k, k' : \mathbb{N}^{p+1} \to \{0,1\}$ be computable in* **FACC**[**2**]*, then computation through the schema defined by the IVP below:*

$$\begin{aligned} f(0,\mathbf{y}) &= g(\mathbf{y}) \\ \frac{\partial f(x,\mathbf{y})}{\partial \ell} &= -f(x,\mathbf{y}) \times k(x,\mathbf{y}) + k(x,\mathbf{y}) \times k'(x,\mathbf{y}) \end{aligned}$$

is in **FACC**[**2**].

Proof. By definition of ℓ-ODE,

$$f(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} \left(\prod_{t=u+1}^{\ell(x)-1} \Big(1 - k(\alpha(t),\mathbf{y})\Big) \right) \times k'(\alpha(u),\mathbf{y}) \times k(\alpha(u),\mathbf{y}).$$

Notice that $f(x,\mathbf{y}) \in \{0,1\}$. Clearly, $\prod_{t'=u'+1}^{\ell(x)-1} \big(1 - k(\alpha(t'),\mathbf{y})\big) = 1$ when $k(\alpha(t'),\mathbf{y}) = 0$ for all $t' \in \{u'+1, \ldots, \ell(x)-1\}$ and $\prod_{t'=u'+1}^{\ell(x)-1}(1 - k(\alpha(t'),\mathbf{y})) \times k'(\alpha(u'),\mathbf{y}) \times k(\alpha(u'),\mathbf{y}) = 1$ when, in addition, $k'(\alpha(u')) \times k(\alpha(u')) = 1$. Then, for all $u'' \in \{u'+1, \ldots, \ell(x)-1\}$, $\prod_{t=u''+1}^{\ell(x)-1}(1 - k(\alpha(t),\mathbf{y}))\big) \times k(\alpha(u''),\mathbf{y}) \times k'(\alpha(u''),\mathbf{y}) = 0$ (since $k(\alpha(u''),\mathbf{y}) = 0$).
We can compute this value using constant-depth circuits including Mod 2 gates:

- In parallel, we compute $g(\mathbf{y})$ and, for any $z \in \{0, \ldots, \ell(x)-1\}$, the $\ell(x)$ pairs of values $k(\alpha(z))$ and $k'(\alpha(z'))$ (both in $\{0,1\}$). This can be done in **FACC**[**2**] by hypothesis.
- Again in parallel, compute consistency between each pair, i.e. whether $k(\alpha(z)) = k'(\alpha(z))$ and between pairs of subsequent values, i.e. whether $k(\alpha(z)) = k(\alpha(z+1))$
- In the subsequent layer, we consider computation through $\ell(x)$ Mod 2 gates "counting" whether the number of changes of values from the considered $z \in \{0, \ldots, \ell(x)-1\}$ to $\ell(x) - 1$ is even or odd.
- In constant depth, for any $z \in \{0, \ldots, \ell(x) - 1\}$, we consider the disjunction of:
  - the (unbounded fan-in) conjunction of the result of the corresponding consistency gate (i.e. checking whether $k(\alpha(z),\mathbf{y}) = k'(\alpha(z),\mathbf{y})$), of $k(\alpha(z),\mathbf{y})$ and of the negation of the Mod 2 gate for z and
  - the (unbounded fan-in) conjunction of the result of the corresponding consistency gate (i.e. checking whether $k(\alpha(z),\mathbf{y}) = k'(\alpha(z),\mathbf{y})$), of the negation of $k(\alpha(z),\mathbf{y})$ and of Mod 2 gate for z.
- We conclude in one layer computing the (unbounded fan-in) disjunction of all the $\ell(x)$ results of the previous gates (at most one of which can be 1).

□

4.6 Non-Strict Schemas in FACC[2]

Proof of Lemma 5. Let us start with the simple case $s = \mathsf{sg}(f(x, \mathbf{y}))$. Since $K \in \{0, 1\}$,

$$K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) = K_1(x) \times \mathsf{sg}(f(x, \mathbf{y})) + K_0(x) \times \mathsf{cosg}(f(x, \mathbf{y})).$$

Relying on this fact, it is possible to rewrite ℓ-b$_0$ODE as the solution of the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that:

$$\begin{aligned}
\frac{\partial f(x, \mathbf{y})}{\partial \ell} &= f(x, \mathbf{y}) \times (K_0(x) \neq K_1(x) - 1) \\
&\quad + \Bigg(\Big(\big(K_0(x) = K_1(x) = 1\big) \times \textsc{Mod}\ 2 \Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big) \Big) = 0 \Big) \\
&\quad + \Big(\big(K_0(x) = K_1(x) = 0\big) \times \textsc{Mod}\ 2 \Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big) \Big) = 1 \Big) \Bigg) \\
&= -f(x, \mathbf{y}) \times \big(K_0(x) = K_1(x)\big) + \big(K_0(x) = K_1(x)\big) \\
&\quad \times \Bigg(\Big(\big(K_0(x) = 1\big) \times \textsc{Mod}\ 2 \Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big) \Big) = 0 \Big) \\
&\quad + \Big(\big(K_0(x) = 0\big) \times \textsc{Mod}\ 2 \Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big) \Big) = 1 \Big) \Bigg)
\end{aligned}$$

Intuitively, if $K_0(t) \neq K_1(t)$, no change occurs, i.e. $f(t+1) = f(t)$. On the contrary, if $K_0(x) = K_1(x)$ we "remove" previous $f(t, \mathbf{x})$ and return 1 if either

- $K_0(t) = K_1(t) = 1$, i.e. $K(t)$ "generates" a 1, and the number of subsequent changes of values (i.e. $K_0(t') \neq K_0(t'+1)$) such that 1 (alternating) propagates to the final value (assuming t is the last value in $\{0, \ldots, \ell(x) - 1\}$ such that $K_0(t) = K_1(t)$) or
- $K_0(t) = K_1(x) = 0$, i.e. $K(t)$ "generates" a 0, and the subsequent number of changes of values is such that 0 does not propagate to the final value,

and 0 otherwise. It is easy to see that the value computed in this way for $t \in \{0, \ldots, \ell(x)-1\}$ corresponds to the final value of the computation only when for any subsequent $t' \in \{t, \ldots, \ell(x)\}$, $K_0(t') \neq K_1(t')$. Otherwise, the final value is given by the last t' such that $K_0(t') = K_1(t')$. Since now no call to $f(x, \mathbf{y})$ occurs in the lienar equation, this IVP comes out to be an instance of the schema proved in **FACC**[**2**] in Lemma 6, such that, in particular, $k(x, \mathbf{y})$ is $K_0(x) = K_1(x)$ and, clearly, both $k, k' \in \{0, 1\}$ and can be computed in **FACC**[**2**] (as $\textsc{Mod}\ 2 \sum_{i=x}^{\ell(x)} (K_0(i) = K_0(i+1))$ corresponds to computation through $\textsc{Mod}\ 2$ gates.

As mentioned, this proof can be generalized to avoid the limitation $s = \mathsf{sg}(f(x, \mathbf{y}))$, allowing linear expression under the scope of the sign function. Let us consider the occurrence of one such expression in K, namely $s = \mathsf{sg}\big(E(x, \mathbf{y}, f(x, \mathbf{y}))\big)$. Since $K \in \{0, 1\}$ (and $A = -1$), the proof is similar to the previous one except for the fact that, in this case,

for any $t \in \{0, \ldots, \ell(x) - 1\}$, we compute (in parallel) even the sign of $s_0(t), s_1(t)$, and we search for the last $s_0(t) = s_1(t)$. Informally, the corresponding circuit is implemented as follows:

- For any $t \in \{0, \ldots, \ell(x) - 1\}$, compute in parallel $K_0(t), K_1(t)$ and the corresponding $s_0(t), s_1(t)$ (where, as usual, $s_0 = \mathsf{sg}(E(\alpha(t), \mathbf{y}, 0))$ and $s_1 = \mathsf{sg}(E(\alpha(t), \mathbf{y}, 1))$, i.e. 0 or 1 substitutes $f(\alpha(t-1), \mathbf{y}) = K(t-1)$ in s).
- We search for the last $t \in \{0, \ldots, \ell(x) - 1\}$ such that $s_0(t) = s_1(t)$ (if for any $t \in \{0, \ldots, \ell(x) - 1\}$, $s_0(t) \neq s_1(t)$, we consider $g(\mathbf{y})$).
- The construction is as before, but, here, we count whether the (number of) changes of values between $s_0(t')$ and $s_0(t' + 1)$, occurring for $t' \in \{t, \ldots, \ell(x) - 1\}$, is even or odd.
- We select the correct final value, between $K_0(\ell(x) - 1)$ and $K_1(\ell(x) - 1)$ accordingly.

□

Since ℓ-b$_0$ODE has been shown to capture computations performed by Mod 2 gates, our characterization for **FACC**[**2**] is obtained by simply endowing $\mathbb{ACDL}$ with it.

Theorem 2. $\mathbf{FACC}[\mathbf{2}] = [0, 1, \ell, \mathsf{sg}, +, -, \div 2, \pi_i^p, \#; \circ, \ell\text{-ODE}_1, \ell\text{-ODE}_3, \ell\text{-b}_0\text{ODE}]$

Proof. ($\supseteq$) Following the proofs [2, Prop. 12, 17] plus Lemma 5 for ℓ-b$_0$ODE. ($\subseteq$) Consequence of [2, Th. 18] and of the fact that the parity function can be simulated by ℓ-b$_0$ODE (see Example 3). □

4.7 ODE-Based Characterizations of FACC[2]

Alternative Proof of Theorem 2.($\supseteq$). We provide an alternative, indirect proof of $\supseteq$. The proof relies on Clote and Takeuti's characterization of **FACC**[**2**] in terms of BRN [16]:

$$\mathbf{FACC}[\mathbf{2}] = [0, I, s_0, s_1, \ell, BIT, \#; \circ, \text{CRN}, 1\text{-BRN}]$$

As said, all basic functions, $\circ$ and CRN can be rewritten in $\mathbb{ACDL}$, see [2]. It remains to show that a function $f(x, \mathbf{y})$ defined by 1-BRN from g and h as follows:

$$\begin{aligned} f(0, \mathbf{y}) &= g(\mathbf{y}) \\ f(x, \mathbf{y}) &= h\big(x, \mathbf{y}, f(\ell(x), \mathbf{y})\big) \end{aligned}$$

with $f(x, \mathbf{y}) \leq 1$, can be rewritten as the solution of the IVP below:

$$\begin{aligned} \mathsf{1brn}(0, \mathbf{y}) &= g(\mathbf{y}) \\ \frac{\partial \mathsf{1brn}(x, \mathbf{y})}{\partial \ell} &= -\mathsf{1brn}(x, \mathbf{y}) + \mathsf{sg}\big(f(x, \mathbf{y})\big) \times h\big(\ell(x), \mathbf{y}, 1\big) + \mathsf{cosg}\big(f(x, \mathbf{y})\big) \times h\big(\ell(x), \mathbf{y}, 0\big) \end{aligned}$$

which is an instance of (limited) ℓ-b$_0$ODE. □

4.8 The Class $\mathbf{FTC}^0$

In this section, we introduce schemas the computation of which is proved to be in $\mathbf{FTC}^0$. One is defined in terms of strict linear ℓ-ODE and provides a new, compact characterization for this class. The second schemas considered are not strict, as they include simple calls to $f(x, \mathbf{y})$, and are defined uniformly with respect to corresponding (i.e., non-strict) schemas for $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$ (as well as for $\mathbf{FNC}^1$ and $\mathbf{FAC}^1$, see Sections 4.10 and 4.12)

We start by dealing with the linear ODE schema defined deriving along ℓ and so that $A(x, \mathbf{y})$ and $B(x, \mathbf{y})$ are both sg-polynomial expressions, with no call to $f(x, \mathbf{y})$.

Definition 9 (Schema ℓ-pODE)**.** Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-pODE from g and h, if it is the solution of the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = A\big(x, \mathbf{y}, h(x, \mathbf{y})\big) \times f(x, \mathbf{y}) + B\big(x, \mathbf{y}, h(x, \mathbf{y})\big)$$

where A and B are sg-polynomial expressions.

This compactly encapsulates not only iterated multiplication and addition but also ℓ-ODE_1 and ℓ-ODE_3 computation.

Example 4 (Counting and Iterated Addition in ℓ-pODE)**.** Notably, BCount (resp. iterated addition, ItAdd) can be expressed as a special case of ℓ-pODE such that $A = 0$ and $B = k(x, \mathbf{y})$ with $k, g \leq 1$, i.e. by considering an IVP such that $f(0, \mathbf{y}) = g(\mathbf{y})$ and:

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = k(x, \mathbf{y}) \qquad (\text{resp. } \frac{\partial f(x, \mathbf{y})}{\partial \ell} = B(x, \mathbf{y}, h(x, \mathbf{y})).$$

Taking $g(\mathbf{y}) = \mathsf{BIT}(0, y)$ and $k(x, \mathbf{y}) = \mathsf{BIT}(\ell(x), y)$, $f(x, x) = \sharp\{u : \mathsf{BIT}(u, x) = 1\}$.

These examples are especially relevant as they reveal that ℓ-pODE is really more expressive than all strict schemas introduced so far (BCount is not computable in $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$, see [31, 32]). The closure property can be shown to hold for ℓ-pODE by relying on the following result.

Proposition 1. *If $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ are computable in $\mathbf{FTC}^0$ and $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-pODE from g and h, then f is computable in $\mathbf{FTC}^0$ as well.*

Proof. By definition of ℓ-ODE, such f satisfies $f(0, \mathbf{y}) = g(\mathbf{y})$ and

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} \left(\prod_{t=u+1}^{\ell(x)-1} \Big(1 + A(r, \mathbf{y})\Big) \right) \times B(u, \mathbf{y})$$

with the convention that $\prod_z^{z-1} \kappa(x) = 1$ and $B(-1, \mathbf{y}) = g(\mathbf{y})$. Computing this expression can be done combining iterated addition and multiplication, which are in $\mathbf{FTC}^0$. □

As an immediate byproduct we obtain a new, concise ODE-based characterization of $\mathbf{FTC}^0$.

Theorem 3. $\mathbf{FTC}^0 = [0, 1, \ell, \mathsf{sg}, +, -, \div 2, \pi_i^p, \#; \circ, \ell\text{-pODE}]$.

Proof. ($\supseteq$) By Proposition 1. ($\subseteq$) Consequence of the facts that (i) ℓ-pODE is strong enough to express ℓ-ODE$_1$ and ℓ-ODE$_3$, (see Appendix 4.9), together with $\mathbf{FAC}^0 = \mathcal{A}_0 = \mathbb{ACDL}$ (see [2, 12]), and (ii) ℓ-pODE captures BCount computation (see Example 4), which is equivalent to Maj or $\times$ (see [33]), together with the fact that $[0, \pi_i^p, \mathsf{s}_0, \mathsf{s}_1, \ell, \mathsf{BIT}\times, \#; \circ, \text{CRN}] = \mathbf{FTC}^0$ (see [16]). established by [16].

□

4.9 Strict Schemas in $\mathbf{FTC}^0$

As mentioned the closure property can be shown for the linear ODE schema defined deriving along ℓ and considering $A(x, \mathbf{y})$ and $B(x, \mathbf{y})$ to be both polynomial expressions. This is obtained relying on the fact that iterated multiplication and addition can be done in $\mathbf{FTC}^0$.

Lemma 7 (Strict Sharply Bounded Sum)**.** *Let B be a* sg *polynomial expression and $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be functions computable in $\mathbf{FTC}^0$. Then, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined by h and g as the solution of the IVP made of $f(x, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = B(x, \mathbf{y}, h(x, \mathbf{y}))$$

is in $\mathbf{FTC}^0$ as well.

Proof Sketch. By definition of ℓ-ODE, the solution of this system is of the form

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} B(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}))$$

where $\alpha(u) = 2^u - 1$ and with the convention that $B(\cdot, \alpha(-1), \mathbf{y}) = f(x, \mathbf{y}) = g(\mathbf{y})$, and the fact that iterated addition is in $\mathbf{FTC}^0$ (see [33]). □

Lemma 8 (Strict Sharply Bounded Product)**.** *Let A be a* sg *polynomial expression and $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be functions computable in $\mathbf{FTC}^0$. Then, the function $f(x, \mathbf{y})$ defined from $h(x, \mathbf{y})$ and $g(x, \mathbf{y})$ as the solution of the IVP made of $f(0, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = A\big(x, \mathbf{y}, h(x, \mathbf{y})\big) \times f(x, \mathbf{y})$$

is in $\mathbf{FTC}^0$.

Proof Sketch. By definition of ℓ-ODE, the solution of the system is of the form

$$f(x,\mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} (1 + A(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}))$$

with the convention that $\prod_{u=-1}^{x}\kappa(x) = f(0,\mathbf{y}) = g(\mathbf{y})$. The proof is analogous to the one for Lemma 7, here remarking that iterated multiplication is in $\mathbf{FTC}^0$ (see [24]). □

The two results above also hold when dealing with vectors rather than functions.

Proposition 2. *Let $A(x,\mathbf{y})$ and $B(x,\mathbf{y})$ be polynomial expressions. Let $z \in \mathbb{N}$ be such that $z \leq log^c(x)$, for some $c \in \mathbb{N}$ and*

$$f(x,\mathbf{y}) = \sum_{u=-1}^{z-1}\left(\prod_{t=u+1}^{z-1}\Big(1 + A(r,\mathbf{y})\Big)\right) \times B(u,\mathbf{y})$$

with the convention that $\prod_{z}^{z-1}\kappa(x) = 1$ and $B(-1,\mathbf{y}) = T(0,\mathbf{y})$. Then, the function which, given $x,\mathbf{y}$ and $f(x,\mathbf{y})$ as input, outputs $f(x,\mathbf{y})$ can be computed in $\mathbf{FTC}^0$.

Remark 4 (Relationship with ℓ-ODE$_1$ and ℓ-ODE$_3$)**.** Observe that ℓ-ODE$_1$ and ℓ-ODE$_3$ are nothing but special cases of ℓ-pODE: ℓ-ODE$_1$ is nothing but an instance of ℓ-pODE such that $A = 1$ and $B = k(x,\mathbf{y}) \in \{0,1\}$ and ℓ-ODE$_3$ is a special case of ℓ-pODE such that $A = (\div 2) - 1$ and $B = 0$.

In other words, ℓ-pODE encapsulates the computing power of ℓ-ODE$_1$ and ℓ-ODE$_3$ simultaneously. As an immediate consequence $\mathbf{FAC}^0 \subseteq \mathbf{FTC}^0$.

Additionally, in order to better and uniformly understand upper bounds in circuit computation, we consider natural constraints over the non-strict linear length-ODE schema obtained by allowing *simple* calls of $f(x,\mathbf{y})$. As shown throughout the paper, this kind of feature may increase the computational power of a schema, and, here, to ensure that computation remains in $\mathbf{FTC}^0$ we have to restrict the forms of A and B.

Lemma 9. *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be in $\mathbf{FTC}^0$. Then, a function f defined by the IVP with initial value $f(0,\mathbf{y}) = g(\mathbf{y})$ and equation of one of the following forms:*

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell} = K\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big) \text{ or } \frac{\partial f(x,\mathbf{y})}{\partial \ell} = K\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big) \times f(x,\mathbf{y})$$

where $K \in \{0,1\}$ is a sg-polynomial expression and $f(x,\mathbf{y})$ only occurs in expressions of the form $s = \mathsf{sg}(f(x,\mathbf{y}))$ in it, is in $\mathbf{FTC}^0$ as well.

Proof. For the first equation type, notice that, if $g(\mathbf{y}) \geq 1$, then, for any $t \in \{0,\ldots,x\}$, $f(t,\mathbf{y}) \geq 0$ (by definition of the schema, in which $A = 0$ and $B \in \{0,1\}$). Thus, in

this case, $f(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} K_1(u)$ with the convention that $K_1(-1) = g(\mathbf{y})$, and we can conclude by noticing that BCount is in $\mathbf{FTC}^0$. Otherwise, we compute in parallel $g(\mathbf{y})$ and $K_0(t), K_1(t)$ for all $t \in \{0,\ldots,\ell(x)\}$. Then, we search for the first t such that $K_0(t) \neq 0$ and compute $\sum_{u=t}^{\ell(x)-1} K_1(u)$: this would be the expected result since, before t, each summand is equal to 0 and, after t, all signed terms remain strictly positive. Again, this corresponds to BCount, which can be done in $\mathbf{FTC}^0$. In terms of circuit computation, this is implemented as follows:

- In parallel, the circuit computes the value of $g(\mathbf{y})$ and, for each $u \in \{0,\ldots,\ell(x)-1\}$ of both $K_0(u)$ and $K_1(u)$ (this can be done in $\mathbf{FTC}^0$ for hypothesis).
- In one layer, for any $t \in \{0,\ldots,\ell(x)-1\}$, it computes the sum $K_0(t)+\sum_{u=t+1}^{\ell(x)-1} K_1(u)$ by t Count gate.
- In constant depth, for any $t \in \{0,\ldots,\ell(x)-1\}$, it computes the conjunction between $\neg K_0(0), \neg K_0(1), ..., \neg K_0(t-1)$ and $K_0(t)$ and select the corresponding Count gate.

For the second equation type, first note that, if $g(\mathbf{y}) = 0$, trivially $f(x,\mathbf{y}) = 0$ (which is clearly computable in $\mathbf{FTC}^0$). Let $g(\mathbf{y}) \geq 1$. Then, for any $t \in \{0,\ldots,\ell(x)-1\}$, $\mathsf{sg}(f(0,\mathbf{y})) > 0$. Therefore, by definition of ℓ-ODE, $f(x,\mathbf{y}) = \prod_{u=-1}^{\ell(x)-1}\big(1+K_1(u)\big)$, with $\prod_{t=-1}^{t'} \kappa = g(\mathbf{y})$. This can be computed in $\mathbf{FTC}^0$: $g(\mathbf{y})$ and each $K_1(u')$, for $u' \in \{0,\ldots,\ell(x)-1\}$ can be computed in parallel in the beginning ($g(x,\mathbf{y})$ and $h(x,\mathbf{y})$ being computable in $\mathbf{FTC}^0$ for hypothesis and K_1 being a sg-polynomial expression by definition) and ItMult (so, in particular, iterated multiplication by 1 and 2) is in $\mathbf{FTC}^0$, see [24]. □

4.10 The Class $\mathbf{FNC}^1$

In this section, we consider two restrictions on ℓ-ODE, which give us natural schemas the solution of which is a function computable in $\mathbf{FNC}^1$: namely, the schema defined by $A = 1$ and $B = K \in \{0,1\}$ and the one defined by $A = -1$ and more liberal $B \geq 0$ (obtaining the so-called ℓ-bODE schema, somewhat generalizing ℓ-b$_0$ODE).

Lemma 10 (Bounded Concatenation with Simple Calls)**.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FNC}^1$. Then, computation through the schema defined by the IVP with initial value $f(0,\mathbf{y}) = g(\mathbf{y})$ and such that:*

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell} = f(x,\mathbf{y}) + K\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big)$$

where $K \in \{0,1\}$ is a sg-polynomial expression, and $f(x,\mathbf{y})$ appears in it only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x,\mathbf{y}))$, is in $\mathbf{FNC}^1$.

Proof Sketch. By definition of ℓ-ODE, $f(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} 2^{\ell(x)-u-1} \times K(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}), f(\alpha(u),\mathbf{y}))$, with the convention that $K(\alpha(-1)) = g(\mathbf{y})$. Let us define the truncated sum (or concatenation) between a and $b-1$ bounded by $\ell(x)-1$, as $S(a,b) =$

$\sum_{u=a}^{b-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), (u)\big)$, with $S(u) = S(\alpha(u), \mathbf{y}) = f(\alpha(u), \mathbf{y})$. In particular, we consider two sums in which the sign of the first term is assumed to be either 0 or 1: $S_0(a, b) = \sum_{u=a}^{b-1} K(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), S(u))$, with $S_0(a) = 0$ and $S_1(a, b) = \sum_{u=a}^{b-1} 2^{\ell(x)-u-1} \times K_1(u)$. Therefore, for any x and $\mathbf{y}$:

$$f(x, \mathbf{y}) = S(-1, \ell(x)/2) + S(\ell(x)/2, \ell(x)) = + \begin{cases} S(-1, \ell(x)/2) \\ S_0(\ell(x)/2, \ell(x)) \times (1 - s(\ell(x)/2 - 1)) \\ S_1(\ell(x)/2, \ell(x)) \times s(\ell(x)/2 - 1). \end{cases}$$

Clearly, $S(-1, \ell(x)/2), S_0(\ell(x)/2, \ell(x))$ and $S_1(\ell(x)/2, \ell(x))$ can be computed in parallel: we know the actual starting value of $S_0(\ell(x)/2, \ell(x))$, which is 0, while to compute $S_1(\ell(x)/2, \ell(x)) = \sum_{u=\ell(x)/2}^{\ell(x)-1} 2^{\ell(x)-u-1} K_1(u)$ it is enough to know that, for any $a > \ell(x)/2$, $f(\alpha(a), \mathbf{y}) > 0$. Once $S(-1, \ell(x)/2) = f(\alpha(\ell(x)/2), \mathbf{y})$ is computed and its sign tested, we can select between S_0 and S_1 accordingly. We conclude by taking the concatenation between $S(-1, \ell(x)/2)$ and either S_0 or S_1. □

This schema can be generalized so to allow calls to $f(x, \mathbf{y})$ in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}) - c)$, i.e. testing whether $f(x, \mathbf{y}) \geq c$. Computation through an extended version in which $f(x, \mathbf{y})$ occurs under the scope of bit is provably in $\mathbf{FNC}^1$ but it is also strong enough to capture binary tree structure, hence to provide a direct encoding of $\mathbf{FNC}^1$ circuit computation.

Remark 5. The condition that f appears only in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$ or even $\mathsf{sg}(f(x, \mathbf{y}) - c)$ might seem limited. However the possibilities it offers really increase the ODE's expressive power and, on the other side, even for slightly more liberal conditions, provide the only known upper bound for the schema is **FP**.

4.11 Schemas in $\mathbf{FNC}^1$

Proof of Lemma 10. By definition of ℓ-ODE,

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} 2^{\ell(x)-u-1} \times K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})\big)$$

with the convention that $K(\alpha(-1), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})) = f(0, \mathbf{y}) = g(\mathbf{y})$. Let us consider the truncated sum (or concatenation) between a and $b - 1$ bounded by $\ell(x) - 1$, which is defined as:

$$S(a, b) = \sum_{u=a}^{b-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), S(u), \mathbf{y})\big) = \sum_{u=a}^{b-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), s(u)\big)$$

with $S(a,\mathbf{y}) = f(\alpha(a),\mathbf{y})$ and $s(a) = \mathsf{sg}\big(f(\alpha(a),\mathbf{y})\big)$. In particular, we consider the following two sums in which the sign of the first term is assumed to be either 0 or 1:

$$S_0(a,b) = \sum_{u=a}^{b-1} K(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S_0(u))$$
$$S_1(a,b) = \sum_{u=a}^{b-1} 2^{\ell(x)-u-1}K_1(u)$$

with $S_0(u) = 0$. Clearly, if $K(a) = K_1(a)$, $f(\alpha(a),\mathbf{y})$ must be strictly greater than 0 and, then, by definition, for any $t \in \{\alpha(a),\ldots,x\}$, $f(t,\mathbf{y}) > 0$. Therefore, for any x and $\mathbf{y}$, the value of $f(x,\mathbf{y})$ can be computed as follows:

$$\begin{aligned} f(x,\mathbf{y}) &= S(-1,\ell(x)) \\ &= S(-1,\ell(x)/2) + S(\ell(x)/2,\ell(x)) \\ &= f(\alpha(\ell(x)/2),\mathbf{y}) + S(\ell(x)/2,\ell(x)) \\ &= \begin{cases} 0 + S_0(\ell(x)/2,\ell(x)) \\ S(-1,\ell(x)/2) + \sum_{u=a}^{b-1} 2^{\ell(x)-u-1}K_1(u) \end{cases} \end{aligned}$$

Hence,

$$f(x,\mathbf{y}) = + \begin{cases} S(-1,\ell(x)/2) \\ S_0(\ell(x)/2,\ell(x)) \times 1 - s(\ell(x)/2-1) \\ S_1(\ell(x)/2,\ell(x)) \times s(\ell(x)/2-1) \end{cases}$$

Independently computing $S(-1,\ell(x)/2), S_0(\ell(x)/2,\ell(x))$ and $S_1(\ell(x)/2),\ell(x))$ can be done in parallel: as seen, we know the actual starting value of $S_0(\ell(x)/2,\ell(x))$, which is $f(\alpha(\ell(x)/2),\mathbf{y}) = 0$ (so that this sum can be computed exactly as the starting sum $S(-1,\ell(x)/2)$ in at most $\ell(x)$ steps), while to compute $S_1(\ell(x)/2,\ell(x)) = \sum_{u=\ell(x)/2}^{\ell(x)-1} 2^{\ell(x)-u-1}K_1(u)$ we do not need to know the actual value of $f(\alpha(\ell(x)/2),\mathbf{y})$ since we know that it is always strictly greater than 0, i.e. for $t \in \{a,\ldots,x\}$ $K(t) = K_1(t)$. Once that $S(-1,\ell(x)/2) = f(\alpha(\ell(x)/2),\mathbf{y})$ is known, the sign can be tested and selection between S_0 and S_1 obtained accordingly. The final step amounts in a concatenation between $S(-1,\ell(x)/2)$, which is either 0 or has length strictly smaller than $\ell(x)/2$, and S_0 or S_1. □

Lemma 11 (Generalized Bounded Concatenation with Simple Calls)**.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be functions computable in $\mathbf{FNC}^1$. Then, $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined by h and g as follows:*

$$\begin{aligned} f(0,\mathbf{y}) &= g(\mathbf{y}) \\ \frac{\partial f(x,\mathbf{y})}{\partial \ell} &= f(x,\mathbf{y}) + K\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big) \end{aligned}$$

with $K \in \{0,1\}$ and f appearing only under the scope of the sign function inside expressions of the form $s = \mathsf{sg}(f(x,\mathbf{y}) - c)$ for any constant $c \in \mathbb{N}$ is in $\mathbf{FNC}^1$.

Proof. By definition of ℓ-ODE,

$$f(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} 2^{\ell(x)-u-1} \times K\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),f(\alpha(u),\mathbf{y})\big)$$

with the convention that $K\big(\alpha(-1),\mathbf{y},h(\alpha(u),\mathbf{y}),f(\alpha(u),\mathbf{y})\big) = f(0,\mathbf{y}) = g(\mathbf{y})$. Let $s_j(u) = \mathsf{sg}\big(f(\alpha(u),\mathbf{y})\big)$, $j \in \{1,\ldots,h\}$ be a list of signed terms appearing in K and $c = \max_j c_j$. Let us consider the truncated sum between a and $b-1$ bounded by $\ell(x)-1$. It is defined as:

$$S(a,b) = \sum_{u=a}^{b-1} K\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S(u)\big)$$

with $S(a) = f\big(\alpha(a),\mathbf{y}\big)$. We consider the following $c+2$ sums in which an assumption is made on the value of the first term:

$$S_i(a,b) = \sum_{u=a}^{b-1} 2^{\ell(x)-u-1} K_i\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S_i(\alpha(u),\mathbf{y})\big)$$

for $i \in \{1,\ldots,c+1\}$. Recall that K_i is defined from K by replacing each term $s_j(a)$ by its value assuming $S_0(a) \leq 0, S_i(u) = i$ for $i \in \{1,\ldots c\}$ and $S_{c+1}(\alpha(a),\mathbf{y}) > c$. More precisely, for any i, $s_j(a) = 1$ if $i > c_j$ and 0 otherwise. Then, for any x and $\mathbf{y}$, the value of $f(x,\mathbf{y})$ can be computed as follows:

$$\begin{aligned}
f(x,\mathbf{y}) &= S(-1,\ell(x)) \\
&= S\big(-1,\ell(x)/2\big) + S\big(\ell(x)/2,\ell(x)\big) \\
&= f\big(\alpha(\ell(x)/2),\mathbf{y}\big) + S\big(\ell(x)/2,\ell(x)\big) \\
&= \vee \begin{cases} 0 + S_0\big(\ell(x)/2,\ell(x)\big) \\ \ldots \\ c + S_c\big(\ell(x)/2,\ell(x)\big) \\ S\big(-1,\ell(x)/2\big) + S_{c+1}\big(\ell(x)/2,\ell(x)\big). \end{cases}
\end{aligned}$$

Hence,

$$f(x,\mathbf{y}) = + \begin{cases} S\big(-1,\ell(x)/2\big) \\ S_0\big(\ell(x)/2,\ell(x)\big) \times \mathsf{sg}\big(S\big(-1,\ell(x)/2\big)\big) \\ S_1\big(\ell(x)/2,\ell(x)\big) \times \mathsf{sg}\big(S\big(-1,\ell(x)/2\big) - 1\big) \times \mathsf{sg}\big(1 - S\big(-1,\ell(x)/2\big)\big) \\ \ldots \\ S_c\big(\ell(x)/2,\ell(x)\big) \times \mathsf{sg}\big(S\big(-1,\ell(x)/2\big) - c\big) \times \mathsf{sg}\big(c - S\big(-1,\ell(x)/2\big)\big) \\ S_{c+1}\big(\ell(x)/2,\ell(x)\big) \times \mathsf{sg}\big(S\big(-1,\ell(x)/2\big) - c - 1\big). \end{cases}$$

Independently computing $S\big(-1,\ell(x)/2\big), S_0\big(\ell(x)/2,\ell(x)\big),\ldots,S_{c+1}\big(\ell(x)/2,\ell(x)\big)$ can be done in parallel: as seen, we know the actual starting value of $S_0\big(\ell(x)/2,\ell(x)\big)$, which is $f\big(\alpha(u),\mathbf{y}\big)=0$ (so that this sum can be computed exactly as the starting sum $S\big(-1,\ell(x)/2\big)$ in at most $\ell(x)$ steps), while to compute each $S_i\big(\ell(x)/2,\ell(x)\big)=\sum_{u=\ell(x)/2}^{\ell(x)-1}2^{\ell(x)-u-1}K_1(u)$ we make the assumption that the actual value of $f(\alpha(a),\mathbf{y})$ □

We now move to the new characterization of $\mathbf{FNC}^1$. To establish it, we introduce an ℓ-ODE schema defined by imposing $A=-1$ and $B\geq 0$, that is by generalizing the constraint defining ℓ-b$_0$ODE from $B=K\in\{0,1\}$ to B taking any positive value.

Definition 10 (Schema ℓ-bODE)**.** Given $g:\mathbb{N}^p\to\mathbb{N}$ and $h:\mathbb{N}^{p+1}\to\mathbb{N}$, the function $f:\mathbb{N}^{p+1}\to\mathbb{N}$ is defined by ℓ-bODE if it is the solution of the IVP with initial value $f(0,\mathbf{y})=g(\mathbf{y})$ and such that

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell}=-f(x,\mathbf{y})+B\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big)$$

where, for any $x,\mathbf{y}$, $B(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y}))\geq 0$ is a sg-polynomial expression, and $f(x,\mathbf{y})$ occurs in it only in expressions of the form $\mathsf{sg}(f(x,\mathbf{y})-c)$, being c a constant.

Lemma 12. *If $f(x,\mathbf{y})$ is defined by ℓ-bODE from functions in $\mathbf{FNC}^1$, then $f(x,\mathbf{y})$ is in $\mathbf{FNC}^1$ as well.*

Proof. To simplify the exposition, w.l.o.g. let us suppose that: (i) B includes only one signed term, say s, in which $f(x,\mathbf{y})$ occurs, and (ii) $\ell(x)$ is a power of two (to avoid rounding). By definition of ℓ-ODE, $f(x,\mathbf{y})=B\big(\alpha(\ell(x)-1),\mathbf{y},h(\alpha(\ell(x)-1),\mathbf{y}),f(\alpha(\ell(x)-1),\mathbf{y})\big)=B_i(\ell(x)-1)$, with $i=s(\ell(x)-1)$. We consider partial or truncated results between a and $b-1$ bounded by $\ell(x)-1$, intuitively corresponding to subsequent compositions of s in B, going from a to $b-1$; that is, either $C(a,b)=C_0(a,b)$, i.e. the computation of $f(b-1,\mathbf{y})$ starting from $f(a,\mathbf{y})=B_0(a-1)$, or $C(a,b)=C_1(a,b)$, i.e. the computation of $f(b-1,\mathbf{y})$ starting from $f(a,\mathbf{y})=B_1(a-1)$. The values of $C(0,a-1)$, $C_0(a,b)$ and $C_1(a,b)$ can be computed independently from all previous values of $f(\cdot,\mathbf{y})$. Additionally, only one between $C_0(a,b)$ and $C_1(a,b)$ corresponds to the actual value of $C(a,b)$. Then, for any x and $\mathbf{y}$, the value of $f(x,\mathbf{y})$ can be computed inductively by selecting the correct partial $B(a,b)$ as follows: $f(x,\mathbf{y})=C(-1,\ell(x))=\mathsf{sg}\big(C(-1,\ell(x)/2)\big)\times C_1\big(\ell(x)/2,\ell(x)\big)+\mathsf{cosg}\big(C(-1,\ell(x)/2)\big)\times C_0\big(\ell(x)/2,\ell(x)\big)$. Both $C(-1,\ell(x)/2), C_0(\ell(x)/2,\ell(x))$ and $C_1(\ell(x)/2,\ell(x))$ as well as, for any $t\in\{0,\ldots,\ell(x)-1\}$, $B_0(t),B_1(t)$ and the corresponding signs can be computed independently from each other and in parallel. The number of subsequent recursive calls needed to end up with the desired selection, i.e. the one corresponding to $C_0(\ell(x)/2,\ell(x))$ and $C_1(\ell(x)/2,\ell(x))$, would require $\ell(x)/2$ (selection) steps, ensuring we are in $\mathbf{FNC}^1$.
□

Remark 6. As desired, Lemma 12 still holds when dealing with $s_0,\ldots,s_r$ signed expressions occurring in B. To deal with this more general case we need to add an initial consistency check, considering $\mathsf{sg}(B_{i_0,\ldots,i_r}(t))$'s and $B_{i'_0\ldots i'_r}(t+1)$'s, in the very beginning.

Together with ℓ-pODE, ℓ-bODE provides a new ODE-based characterization for $\mathbf{FNC}^1$.

Theorem 4. $\mathbf{FNC}^1 = [0, 1, \ell, \mathsf{sg}, +, -, \div 2, \#, \pi_i^p; \circ, \ell\text{-pODE}, \ell\text{-bODE}]$.

Proof. ($\supseteq$) All basic functions of this algebra are already in $\mathbf{FAC}^0$ and the closure property holds for $\circ, \ell$-pODE (generalization of Proposition 1) and ℓ-bODE (Lemma 12).
($\subseteq$) The proof is indirect and passes through Clote's algebra:

$$\mathcal{N}_0' = [0, \pi_i^p, \mathsf{s}_0, \mathsf{s}_1, \ell, \mathsf{BIT}, \#; \circ, \text{CRN}, \text{4-BRN}],$$

where a function is defined by 4-BRN if $f(0, \mathbf{y}) = g(\mathbf{y})$ and $f(s_i(x), \mathbf{y}) = h_i(x, \mathbf{y}, f(x, \mathbf{y}))$ for g and h taking values in $\{0, \dots, 4\}$ only. In [13], this class is proved to be the one characterizing $\mathbf{FNC}^1$. In particular, to show that any function defined by 4-BRN can be rewritten as the solution of an instance of ℓ-bODE, we consider the IVP defined by the initial value $\mathsf{4brn}(x, y, \mathbf{y}) = g(\mathbf{y})$ and such that:

$$\begin{aligned}\frac{\partial \mathsf{4brn}(x, y, \mathbf{y})}{\partial \ell} = -\mathsf{4brn}(x, y, \mathbf{y}) &+ h_{z(x)}(\ell(x), \mathbf{y}, 0) \times \mathsf{cosg}\big(\mathsf{4brn}(x, y, \mathbf{y})\big) \\ &+ h_{z(x)}(\ell(x), \mathbf{y}, 1) \times \mathsf{sg}\big(\mathsf{4brn}(x, y, \mathbf{y})\big) \times \mathsf{cosg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 1\big) \\ &+ h_{z(x)}(\ell(x), \mathbf{y}, 2) \times \mathsf{sg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 1\big) \times \mathsf{cosg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 2\big) \\ &+ h_{z(x)}(\ell(x), \mathbf{y}, 3) \times \mathsf{sg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 2\big) \times \mathsf{cosg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 3\big) \\ &+ h_{z(x)}(\ell(x), \mathbf{y}, 4) \times \mathsf{sg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 3\big) \times \mathsf{cosg}\big(\mathsf{4brn}(x, y, \mathbf{y}) - 4\big)\end{aligned}$$

with $z(x) = \mathsf{BIT}(\ell(y) - \ell(x) - 1, y)$. Then, $\mathsf{4brn}(x, x, \mathbf{z}) = f(x, x, \mathbf{z})$, where f is defined by 4-BRN from g and h_i. Since Clote's $\mathcal{N}_0'$ is nothing but $\mathcal{A}_0$ endowed with 4-BRN [13] and by [2] $\mathcal{A}_0 = \mathbb{ACDL}$, this encoding of 4-BRN via ℓ-bODE, together with Proposition 2, concludes our proof. □

4.12 The Class $\mathbf{FAC}^1$

When we consider a generalization of the non-strict schemas shown in $\mathbf{FTC}^0$ from $B = K \in \{0, 1\}$ to more liberal $B \geq 0$ (analogously to the generalization from ℓ-b$_0$ODE to ℓ-bODE), we obtain an ℓ-ODE schema, which is provably in $\mathbf{FAC}^1$. Formally, it is defined by an IVP with restrictions $A = 0$ and, as mentioned, $B \geq 0$ with only simple calls to $f(x, \mathbf{y})$.

The proof of the result below is similar to that for Lemma 10, except for the last step which amounts of a sum (possibly including carries) rather than a concatenation (see Appendix 4.13).

Lemma 13 (Bounded Sum with Simple Calls)**.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^1$. Then, computation through the schema defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where $B \geq 0$ is a sg-polynomial expression with $f(x, \mathbf{y})$ only appearing under the scope of the sign function inside expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, is in $\mathbf{FAC}^1$ as well.

4.13 Schemas in FAC[1]

Proof of Lemma 13. First observe that, by definition of ℓ-ODE,

$$f(x,\mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} B\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),f(\alpha(u),\mathbf{y})\big)$$

with the convention that $B(\alpha(-1),\mathbf{y},h(\alpha(-1),\mathbf{y}),f(\alpha(-1),\mathbf{y})) = g(\mathbf{y}) = f(0,\mathbf{y})$.
We can consider the truncated sums between a and $b-1$ bounded by $\ell(x)-1$. It is defined by

$$S(a,b) = \sum_{u=a}^{b-1} B\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S(\alpha(u),\mathbf{y})\big) = \sum_{u=a}^{b-1} B\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),s(\alpha(u))\big)$$

with $S(\alpha(a),\mathbf{y}) = f(\alpha(a),\mathbf{y})$ and $s(a) = \mathsf{sg}(f(\alpha(a-1),\mathbf{y}))$. By definition of $f(x,\mathbf{y})$, if for some $t \in \{0,\dots,\ell(x)-1\}$, $f(t,\mathbf{x}) > 0$, then, for all $t' \in \{t+1,\dots,\ell(x)-1\}$, $f(t,\mathbf{x}) > 0$, that is $B(t') = B_1(t')$. Thus, we can consider the following two similar sums:

$$\begin{aligned}
S_0(a,b) &= \sum_{u=a}^{b-1} B\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S_0(\alpha(u),\mathbf{y})\big)\\
S_1(a,b) &= \sum_{u=a}^{b-1} B\big(\alpha(u),\mathbf{y},h(\alpha(u),\mathbf{y}),S_0(\alpha(u),\mathbf{y})\big) = \sum_{u=a}^{b-1} B_1\big(\alpha(u)\big)
\end{aligned}$$

with $S_0(\alpha(a),\mathbf{y}) = 0$ and $S_1(\alpha(a),\mathbf{y}) = 1$. Clearly, $S(a,b)$ is either $S_0(a,b)$ or $S_1(a,b)$. (Notably, in the former case $S(0,b) = S_0(a,b)$.) Then, for any $x,\mathbf{y}$, the value of $f(x,\mathbf{y})$ can be computed inductively as follows:

$$\begin{aligned}
f(x,\mathbf{y}) &= S\big(-1,\alpha(\ell(x))\big)\\
&= S\big(-1,\alpha(\ell(x)/2)\big) + S\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big)\\
&= f\big(\alpha(\ell(x)/2),\mathbf{y}\big) + S\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big)\\
&= \begin{cases} 0 + S_0\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big)\\ S\big(-1,\ell(x)/2\big) + \sum_{u=a}^{b-1} B_1(\alpha(u)) \end{cases}
\end{aligned}$$

Hence,

$$f(x,\mathbf{y}) = + \begin{cases} S(-1,\alpha(\ell(x)/2))\\ S_0\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big) \times 1 - s\big(\alpha(\ell(x)/2)-1\big)\\ S_1\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big) \times s\big(\alpha(\ell(x)/2)-1\big) \end{cases}$$

Computing $S\big(-1,\alpha(\ell(x)/2\big), S_0\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big)$ and $S_1\big(\alpha(\ell(x)/2),\alpha(\ell(x))\big)$ can be done independently in parallel. In particular, as seen, we know that the actual starting

value of $S_0\big(\alpha(\ell(x)/2), \alpha(\ell(x))\big)$ (which is $f(u, \mathbf{y}) = 0$), while for $S_1\big(\alpha(\ell(x)/2), \alpha(\ell(x))\big) = \sum_{u=\ell(x)/2}^{\ell(x)-1} B_1(\alpha(u))$ we do not need to know the actual value of f through the computation, since we know that it is always strictly greater than 0. Once $S(-1, \alpha(\ell(x)/2)) = f(\alpha(\ell(x)/2), \mathbf{y})$ is known, its sign can be tested and the outermost addition can be performed. Testing the sign of $f(t, \mathbf{y})$ can be done in $O(1)$. So, the depth of the process obeys the following recursive equation:

$$depth(\ell(x)) = depth(\ell(x)/2) + O(1).$$

It is known that this given $depth(\ell(x)) = O(\ell_2(x))$, hence is logarithmic in the length $\ell(x)$ of one of the input. □

5 Characterizing Classes Deriving along ℓ_2

Here, we consider alternative linear ODEs in which derivation is not along ℓ but along the "slower" function ℓ_2. Of course, in this context, same syntactic constraints on the linearity of the schema end up characterizing smaller classes (for example, the solution of the system shown in $\mathbf{FAC}^1$ when deriving along ℓ is shown to be computable in $\mathbf{FTC}^0$ if derivation is along ℓ_2). The main result of this section is that linear ODEs with derivative along ℓ_2 are in $\mathbf{FNC}^1$ (rather than $\mathbf{FP}$ as in the case of ℓ [9]).

First, observe that when deriving along ℓ_2, computation through a strict linear ODE schema is (of course in $\mathbf{FTC}^0$ but even) in $\mathbf{FAC}^0$ and corresponds to LogItAdd, i.e. iterated addition of $\ell_2(n)$ numbers.

Lemma 14. *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^0$. Then, $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined from h and g by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = B\big(x, \mathbf{y}, h(x, \mathbf{y})\big)$$

where B is a limited sg*-polynomial expression, is in $\mathbf{FAC}^0$.*

When this schema is generalized to the non-strict setting by allowing $f(x, \mathbf{y})$ to occur in B under the scope of the sign function, computation is proved to be in $\mathbf{FTC}^0$ by a much evolved argument.

Lemma 15. *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be definable in $\mathbf{FTC}^0$. Then, computation through the schema defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where B is a sg*-polynomial expression in which $f(x, \mathbf{y})$ only appears under the scope of the sign function, is in $\mathbf{FTC}^0$.*

Lemma 16. *Let B be a limited* sg *polynomial expressions and $g : \mathbb{N}^p \to \mathbb{N}, h : \mathbb{N}^{p+1} \to \mathbb{N}$ be functions computable in* $\mathbf{FAC}^0$*. Then, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined from B, h and g by the IVP below:*

$$f(0, \mathbf{y}) = g(\mathbf{y})$$
$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = B\big(x, \mathbf{y}, h(x, \mathbf{y})\big)$$

is in $\mathbf{FAC}^0$*.*

Proof. By definition of ℓ_2-ODE, the solution of the given system is of the form:

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell_2(x)-1} B\big(\alpha_2(u), \mathbf{y}, h(\alpha_2(u), \mathbf{y})\big)$$

where $\alpha_2(u) = 2^{2^u-1} - 1$ and with the convention that $B(\alpha_2(-1), \mathbf{y}, h(\alpha_2(-1), \mathbf{y}) = f(0, \mathbf{y}) = g(\mathbf{y})$. Since g, h (by hypothesis) and $+$, -, sg (B is limited sg-polynomial expression) are in $\mathbf{FAC}^0$, each $B\big(\alpha_2(u), \mathbf{y}, h(\alpha_2(u))\big)$ is. There are at most $\ell_2(x)$ terms $B\big(\alpha_2(u), \mathbf{y}, h(\alpha_2(u))\big)$, so their sum, i.e. $f(x, \mathbf{y})$, can be computed in constant time (LogItAdd is $\mathbf{FAC}^0$, see [33]). □

Proof of Lemma 15. To ease the presentation, we make the non-essential assumption that there is only one signed term $s(f(x, \mathbf{y})) = \mathsf{sg}(P(f(x, \mathbf{y})))$ occurring in B, where P is a polynomial expression.
Clearly, each $s(f(x, \mathbf{y}))$ can take two values only, i.e. 0 or 1. Since derivation is along $\ell_2(x)$, the value of $f(x)$ depends on the *distinct* values of ℓ_2 between 0 and x, hence of each $\alpha_2(u) = 2^{2^u-1} - 1$, the greatest integer t such that $\ell_2(t) = u$, for $u \in \{0, \ldots, \ell_2(x) - 1\}$. More precisely,

$$f(x, \mathbf{y}) = f(0, \mathbf{y}) + \sum_{u=0}^{\ell_2(x)-1} B(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})).$$

Notice that, for each $t \in \{0, \ldots, \ell_2(x)\}$ it holds that:

$$f(\alpha_2(t), \mathbf{y}) = f(0, \mathbf{y}) + \sum_{u=0}^{t-1} B\big(\alpha_2(u), \mathbf{y}, h(\alpha_2(u), \mathbf{y}), f(\alpha_2(u), \mathbf{y})\big).$$

Hence, each signed term is obtained through partial sums. To compute $f(x, \mathbf{y})$, we will make assumptions on the values of the intermediate signed terms and check afterwards that the assumptions are consistent. For each value u, consider

$$B_0(u) \text{ and } B_1(u)$$

with $B_i(u)$, for $i \in \{0,1\}$ is $B(\alpha_2(u), \mathbf{y}, h(\alpha_2(u), \mathbf{y}), f(\alpha_2(u), \mathbf{y}))$ where $s(f(\alpha_2(u), \mathbf{y}))$ has been replaced by the corresponding $i \in \{0,1\}$.
Clearly, only one choice of sequence for the value of the signed term is consistent and the following holds:

$$f(x, \mathbf{y}) = f(0, \mathbf{y}) + \sum_{\epsilon \in \{0,1\}^{\ell_2(x)}} \sum_{u=0}^{\ell_2(x)-1} B_{\epsilon_u}(u),$$

where $\epsilon = (\epsilon_0, \ldots, \epsilon_{\ell_2(x)-1}) = (\epsilon_u)$. Let's now consider the partial sums:

$$S_{\epsilon,t} = \sum_{u=0}^{t-1} B_{\epsilon_i}(u)$$

for all $\epsilon \in \{0,1\}^{\ell_2(x)}$ and $t \in \{0, \ldots, \ell_2(x)\}$. The correct choice of ϵ is guided by $s(f(0, \mathbf{y}))$, which must be equal to ϵ_0 and by the fact that from one step to the other, the computed sign of the partial sum, i.e. $s(S_{\epsilon,t})$ is equal to the next choice ϵ_t. Then, to summarize, function $f(x, \mathbf{y})$ satisfies the following equation (mixing Boolean and arithmetic operators: since one choice only is consistent, the external operator can be a disjunction):

$$f(x, \mathbf{y}) = f(0, \mathbf{y}) + \bigvee_{\epsilon \in \{0,1\}^{\ell_2(x)}} \left(\bigwedge_{t=0}^{\ell_2(x)-1} (s(S_{\epsilon,t}) = \epsilon_t) \right) \times S_{\epsilon,\ell_2(x)}.$$

We now describe the construction and its complexity:

- In the first layer, for any possible $t \in \{0, \ldots, \ell_2(x)\}$ compute *in parallel*, $B_i(t)$, that is:
  - $B_0(0)$ and $B_1(0)$
  - $B_0(1)$ and $B_1(1)$
  - ...
  - $B_0(\ell_1(x) - 1)$ and $B_1(\ell_1(x) - 1)$

  In this case, since B is a polynomial expression (i.e. computable in $\mathbf{FTC}^0$), each B_i is obtained from B by substitution of one of its input and the number of values to compute is $O(\ell_2(x))$, the implementation is of polynomial size and constant depth on a circuit with MAJ gates.
- For all $\epsilon \in \{0,1\}^{\ell_2(x)}$ and for all $t \in \{1, \ldots, \ell_2(x)\}$, compute in parallel the sums:

  $$S_{\epsilon,t} = \sum_{u=0}^{t-1} B_{\epsilon_u}(u).$$

  Each $S_{\epsilon,t}$ is an iterated sum of at most $O(\ell_2(x))$ terms each computable by $\mathbf{FTC}^0$ circuit. There are $O(\ell_2(x) \times 2^{\ell_2(x)}) = O(\ell_2(x) \times \ell(x))$ such sums, hence a linear number in the input size. The whole step is in $\mathbf{FTC}^0$

- For all $\epsilon \in \{0,1\}^{\ell_2(x)}$ and $t \in \{1,\ldots,\ell_2(x)\}$ compute the sign $s(S_{\epsilon,t})$. Since $s(S_{\epsilon,t})$ is a signed expression, i.e. is a sign function applied to a polynomial expression and since addition and multiplication are in $\mathbf{FTC}^0$, this step is also in $\mathbf{FTC}^0$.

The sequence of tests to evaluate $\bigwedge_{t=0}^{\ell_2(x)-1}(s(S_{\epsilon,t}) = \epsilon_t)$ and then

$$f(0,\mathbf{y}) + \bigvee_{\epsilon\in\{0,1\}^{\ell_2(x)}} \left(\bigwedge_{t=0}^{\ell_2(x)-1} (s(S_{\epsilon,t}) = \epsilon_t) \right) \times S_{\epsilon,\ell_2(x)}$$

can be wired in a constant depth and polynomial size circuit not even using majority gates. The whole computation is in $\mathbf{FTC}^0$. □

As mentioned, the IVP defined deriving along ℓ_2 and allowing "full linearity" is particularly interesting.

Definition 11 (Schema ℓ_2-ODE)**.** Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ_2-ODE if it is the solution of the IVP with initial value $f(0,\mathbf{y}) = g(\mathbf{y})$ and such that:

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell_2} = A\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big) \times f(x,\mathbf{y}) + B\big(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y})\big).$$

This schema captures the computation of expressions in the following form:

$$f(x,\mathbf{y}) = \sum_{u=-1}^{\ell_2(x)-1} \left(\prod_{z=u+1}^{\ell_2(x)-1} \Big(1 + A\Big(\alpha_2(z),\mathbf{y},h(\alpha_2(z),\mathbf{y}),f(\alpha_2(z),\mathbf{y})\Big)\Big) \right) \times B\Big(\alpha_2(u),\mathbf{y},h(\alpha_2(u),\mathbf{y}),f(\alpha_2(u),\mathbf{y})\Big),$$

where $\alpha_2(v) = 2^{2^v-1} - 1$, i.e. $\ell_2(\alpha_2(v)) = v$ and $\alpha_2(v)$ is the greatest integer t such that $\ell_2(t) = v$.

Lemma 17. *Let f be defined by ℓ_2-ODE from g and h in $\mathbf{FNC}^1$, then f is in $\mathbf{FNC}^1$.*

To prove Lemma 17, we need to introduce some preliminary remarks and notational conventions. The following remark will be largely exploited in the proof of Lemma 17.

Remark 7. Since in the given system A and B are essentially constant in $f(\alpha_2(z),\mathbf{y})$, each sub-term of A and B involving $f(\alpha_2(z),\mathbf{y})$ is a sg-polynomial expression of the form $\mathsf{sg}\big(P(\alpha_2(z),\mathbf{y})\big)$. So the dependency is on Boolean value that themselves depend on $f(\alpha_2(z),\mathbf{y})$.

Again, to ease the presentation we make the following (non essential) assumptions:

- The system is of dimension 1, i.e. over scalars
- In A (resp. B) there is only one sub-expression involving $f(x, \mathbf{y})$ denoted by

$$s_1(f(x,\mathbf{y})) = \mathsf{sg}\big(P_1(x,\mathbf{y},f(x,\mathbf{y}))\big)$$

  resp., $s_2(f(x,\mathbf{y})) = \mathsf{sg}\big(P_2(x,\mathbf{y},f(x,\mathbf{y}))\big)$.

Notation 3. Given $i < \ell_2(x)$, let $t_i = \alpha_2(i)$, i.e. $t_0 = 0$, $t_1 = 2^1 - 1 = 1$, $t_2 = 2^3 - 1 = 7$ and, more generally, $t_i = 2^{2^i-1} - 1$. For $i \in \{0, \ldots, \ell_2(x) - 1\}$, let

$$\mathbf{a}(i) : s_1(f(t_i,\mathbf{y})) \mapsto \{0,1\}$$
$$\mathbf{b}(i) : s_2(f(t_i,\mathbf{y})) \mapsto \{0,1\}.$$

Let $A_0(x,\mathbf{y},h(x,\mathbf{y}))$ or even $A_0(x)$ (resp. $A_1(x,\mathbf{y},h(x,\mathbf{y}))$ or $A_1(x)$) denote $A(x,\mathbf{y},h(x,\mathbf{y}),f(x,\mathbf{y}))$ such that all occurrences of $s_1(f(x,\mathbf{y}))$ in it has been substituted by the value 0 (resp. 1). Let us define $f_{\mathbf{a},\mathbf{b}}$ as follows:

$$f_{\mathbf{a},\mathbf{b}}(t_i) = \sum_{u=-1}^{i-1} \left(\prod_{z=0}^{i-1} \Big(1 + A_{\mathbf{a}(z)}\big(t_z,\mathbf{y},h(t_z,\mathbf{y})\big)\Big)\right) \times B_{\mathbf{b}(u)}\big(t_u,\mathbf{y},h(t_u,\mathbf{y})\big)$$

with the convention that $B_{\mathbf{b}(-1)}(\cdot,\cdot,\mathbf{y}) = f(0,\mathbf{y})$ and $\prod_x^{x-1}\kappa(x) = 1$. Intuitively, it holds that

$$f_{\mathbf{a},\mathbf{b}}(t_i) = f(t_i)$$

for the choice of $\mathbf{a},\mathbf{b}$ that passes the following consistency test: check that, for every $j < i$, $s_1(f_{\mathbf{a},\mathbf{b}}(t_j)) = \mathbf{a}(j)$ and $s_2(f_{\mathbf{a},\mathbf{b}}(t_j)) = \mathbf{b}(j)$ for every $j < i$.

We can now examine how to compute in $\mathbf{FNC}^1$ a function $f(x,\mathbf{y})$ defined by ℓ_2-ODE from functions g and h in $\mathbf{FNC}^1$.

Proof of Lemma 17. By definition of ℓ_2-ODE,

$$f(x,\mathbf{y}) = \sum_{u=-1}^{\ell_2(x)-1} \left(\prod_{z=u+1}^{\ell_2(x)-1} \Big(1 + A\big(\alpha_2(z),\mathbf{y},h(\alpha_2(z),\mathbf{y}),f(\alpha_2(z,\mathbf{y}),\mathbf{y})\big)\Big)\right)$$
$$\times B\big(\alpha_2(u),\mathbf{y},h(\alpha_2(u),\mathbf{y}),f(\alpha_2(u),\mathbf{y})\big)$$

where $\alpha_2(t) = 2^{2^t-1} - 1$.

We show that this function can be computed in $\mathbf{FNC}^1$ by constructing the corresponding log-depth circuit with bounded fan-in gates:

- As a first layer, generate in parallel all possible $(\mathbf{a},\mathbf{b}) \in \{0,1\}^{2\ell_2(x)}$ i.e (reordered) the vector.

$$(\mathbf{a}(0),\mathbf{b}(0) \quad \mathbf{a}(1),\mathbf{b}(1) \quad \ldots \quad \mathbf{a}(\ell_2(x)-1)),\mathbf{b}(\ell_2(x)-1)).$$

  There are $O(\ell^2(x))$ such choices.

- For each such $\mathbf{a}, \mathbf{b}$ computes also in parallel

$$f_{\mathbf{a},\mathbf{b}}(t_1) \quad f_{\mathbf{a},\mathbf{b}}(t_2) \quad \dots \quad f_{\mathbf{a},\mathbf{b}}(t_i) \quad \dots \quad f_{\mathbf{a},\mathbf{b}}(t_m).$$

  Each $f_{\mathbf{a},\mathbf{b}}(t_1)$ is an iterated sum of iterated products and can be computed in $\ell_2(x)$ depth. There are $O(\ell_2(x))$ such values to compute.

- For $\mathbf{a}, \mathbf{b}$, and any $i \in \{1, \dots, \ell_2(x) - 1\}$, compute

$$s_1(f_{\mathbf{a},\mathbf{b}}(t_i)) \quad s_2\big(f_{\mathbf{a},\mathbf{b}}(t_i)\big).$$

  Each of these independent computations can be done with depth at most $\ell_2(x)$.

- Next, for any $i \in \{1, \dots, m-1\}$ local consistency is checked, i.e. it is checked in parallel whether

$$s_1(f_{\mathbf{a},\mathbf{b}}(t_i)) = \mathbf{a}(i) \quad s_2(f_{\mathbf{a},\mathbf{b}}(t_i)) = \mathbf{b}(i).$$

  This is done in constant depth.

- Now the circuit also checks global consistency, that is, for the given fixed choice of $(\mathbf{a}, \mathbf{b})$, it is checked whether every local choice is consistent. It is done in the form of a binary tree. Clearly, there is only one pair $\mathbf{a}, \mathbf{b}$ only one of these checks would be different from 0.
  This is done in $\log_3(x)$.

- At this point the computation concludes returning the value of $f_{\mathbf{a},\mathbf{b}}(x)$ corresponding to the (only) choice of $\mathbf{a}, \mathbf{b}$ leading to a positive global check. Roughly speaking this is done under the form of a binary tree which starts comparing the result of the global check for two (consecutive) pairs $\mathbf{a}', \mathbf{b}'$ and $\mathbf{a}'', \mathbf{b}''$ and push further the correct value ($f_{\mathbf{a}',\mathbf{b}'}(x)$ or $f_{\mathbf{a}'',\mathbf{b}''}(x)$) if one of them has a positive global check or a sequence of 0 of appropriate length if none of them is. At the root of the tree, the correct value of $f(x)$ is outputted.

□

6 Conclusion

The objective of this paper was to start a new and uniform approach for studying circuit complexity through the lens of ODEs. This approach allows to consider bounds on size or depth in circuit computation in terms of constraints on the linearity of the equations defining recursion schemas in two main contexts of derivatives: derivation along ℓ and along ℓ_2. Additionally, we have investigated completeness both improving and placing in a coherent setting the results obtained in [2] for $\mathbf{FAC}^0$ and $\mathbf{FTC}^0$, and providing what, to the best of our knowledge, are the first ODE-based characterizations for $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FNC}^1$. In addition of being a simple and natural way to design algorithms, ODE tools

have also enabled us to obtain relatively simple and self-contained membership proofs and characterizations compared to the often convoluted constructions or external theories referenced in previous literature.

Many directions for future investigation remain open. A natural one would be the study of characterizations for the entire **FAC** and **FNC** hierarchies. For this, one has to identify which ODE features or characteristic would help to jump from one depth level, say $(\log n)^i$, to the next. Furthermore, a possible next step of our research involves defining (strict) ℓ-ODE schemas corresponding to complexity classes with counters, lying between **FACC**[**2**] and $\mathbf{FTC}^0$ (i.e. which correspond to computations executed by unbounded fan-in circuits featuring MOD p gates), ultimately allowing us to better understand their interrelationships and the concept of counting within the framework of small circuits. Another relevant and intriguing direction is the exploration of the expressive power of schemas with derivative along ℓ_2 in particular w.r.t. the problem of capturing complexity classes. Studying the relationship between ℓ and ℓ_2 schemas might also be informative. Classical methods such as changes of variables sometimes help to go from one ODE schema to another but, on the other side, it seems that changing the function along which the derivative is taken gives a different point of view in terms of what can be expressed. However, this study has just been initiated here, and a systematic investigation is left for future work. Other challenging directions for subsequent research include generalizing our approach to the continuous setting by analyzing small circuit classes over the reals, following the path outlined in [6] for **FP** and **FPSPACE**. Developing proof-theoretical counterparts to our ODE-style algebras, in the form of natural rule systems inspired by the ODE design to syntactically characterize the corresponding classes, represents another promising avenue for exploration.

References

[1] B. Allen. Arithmetizing uniform NC. *Ann. Pure Appl. Logic*, 53:1–50, 1991.

[2] M. Antonelli, A. Durand, and J. Kontinen. A new characterization of FAC^0 via discrete ordinary differential equations. In *Proc. MFCS*, 2024.

[3] T. Arai. A bounded arithmetic AID for Frege systems. *Ann. Pure Appl. Logic*, 103:155–199, 2000.

[4] D.A.M. Barrington, N. Immerman, and H. Straubing. On uniformity within NC^1. *J. of Comput. and Syst. Sc.*, 41:274–306, 1990.

[5] S. Bellantoni and S. Cook. A new recursion-theoretic characterization of poly-time functions. *Comput. Complex.*, 2:97–110, 1992.

[6] M. Blanc and O. Bournez. A characterisation of functions computable in polynomial time and space over the reals with discrete ordinary differential equations: simulation of Turing Machines with analytic discrete ODEs. In *Proc. MFCS*, pages 21:1–21:15, 2023.

[7] M. Blanc and O. Bournez. The complexity of computing in continuous time: Space complexity is precision. In *Proc. ICALP*, pages 129:1–129:22, 2024.

[8] G. Bonfante, R. Kahle, J.-Y. Marion, and I. Oitavem. Two function algebras defining functions in NC^k boolean circuits. *Inf. and Comput.*, 2016.

[9] O. Bournez and A. Durand. Recursion schemes, discrete differential equations and characterization of polynomial time computation. In *Proc. MFCS*, 2019.

[10] O. Bournez and A. Durand. A characterization of functions over the integers computable in polynomial time using discrete differential equations. *Comput. Complex.*, 32(7), 2023.

[11] P.G. Clote. A sequential characterization of the parallel complexity class NC. Technical report, Boston College, 1988.

[12] P.G. Clote. *Sequential, machine-independent characterizations of the parallel complexity classes AlogTIME, AC^k, NC^k and NC*, pages 49–69. Progress in Computer Science and Applied Logic. Birkhäuser, Boston, MA, 1990.

[13] P.G. Clote. On polynomial size Frege proofs of certain combinatorial principles. In P. Clote and Krjícek J., editors, *Arithmetic, Proof Theory, and Computational Complexity*, pages 166–184. Clarendon Press, 1993.

[14] P.G. Clote and E. Kranakis. *Boolean Functions and Computation Models*. Springer, 1998.

[15] P.G. Clote and G. Takeuti. Bounded arithmetic for NC, ALogTIME, L and NL. *Ann. Pure and Appl. Logic*, 56:73–117, 1992.

[16] P.G. Clote and G. Takeuti. First order bounded arithmetic and small complexity classes. In P.G. Clote and J.B. Remmel, editors, *Feasible Mathematics II*, pages 154–218. 1995.

[17] A. Cobham. The intrinsic computational difficulty of functions. In *Logic, Methodology and Phylosophy of Science: Proc. 1964 International Congress*, pages 24–30. 1965.

[18] K.J. Compton and C. Laflamme. An algebra and a logic for NC^1. *Inf. Comput.*, 87(1/2):240–262, 1990.

[19] S. Cook and T. Morioka. Quantified propositional calculus and a second-order theory for NC^1. *Arch. Math. Logic*, 44:711–749, 2005.

[20] S. Cook and P. Nguyen. Theories for TC^0 and other small circuit classes. *Log. Meth. Comput. Sci.*, 2(1–40), 2006.

[21] A. Durand, A. Haak, and H. Vollmer. Model-theoretic characterization of Boolean and arithmetic circuit classes of small depth. In *Proc. LICS*, pages 354–363, 2018.

[22] M.L. Furst, J.B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. In *Proc. FOCS*, pages 260–270, 1981.

[23] Y. Gurevich and H. Lewis. A logic for constant-depth circuit. *Inf. Control*, 61:65–74, 1984.

[24] W. Hesse, E. Allender, and D.A.M. Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences*, 65(4):695–716, 2002.

[25] N. Immerman. Languages that capture complexity classes. *SIAM J. Comput.*, 16:760–778, 1987.

[26] J. Johannsen. *A bounded arithmetic theory for constant depth threshold circuits*, volume 6 of *Springer Lecture Notes in Logic*, chapter GÖDEL '96, pages 224–234. Hájek, P., 1996.

[27] D. Leivant. Predicative recurrence and computational complexity I: Word recurrence and poly-time. In *Feasible Mathematics*. Birkhüser, 1994.

[28] D. Leivant and J.-Y. Marion. Ramified recurrence and computational complexity II: Substitution and poly-space. In *Proc. CSL*, pages 369–380, 1995.

[29] D. Leivant and Y.-Y. Marion. Lambda calculus characterizations of poly-time. *Fundam. Inform.*, 19(1,2):167–184, 1993.

[30] S. Lindell. A purely logical characterization of circuit uniformity. In *7th Structure in Complexity Theory Conf.*, pages 185–192, 1992.

[31] A.A Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41:333–338, 1987.

[32] R Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In Alfred V. Aho, editor, *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA*, pages 77–82. ACM, 1987. doi:10.1145/28395.28404.

[33] H. Vollmer. *Introduction to Circuit Complexity: A Uniform Approach*. Springer, 1999.